

SZKOŁA GŁÓWNA HANDLOWA W WARSZAWIE
WARSAW SCHOOL OF ECONOMICS

Studium magisterskie

Kierunek: Analiza Danych – Big Data

Laura Hoang, Antoni Ballaun
Nr albumu: 140381, 140447

Analiza operatorów sieciami neuronowymi

Praca magisterska
napisana w Instytucie Ekonometrii
pod kierunkiem naukowym
dr Sebastiana Zająca

Warszawa, 2026

Wykaz autorów i zakres przygotowanych rozdziałów

Laura Hoang

Wprowadzenie
Rozdział 1.: 1.1, 1.2, 1.3
Rozdział 4.: 4.1.3, 4.2.3
Streszczenie

Antoni Ballaun

Rozdział 2.: 2.1, 2.2, 2.3
Rozdział 3.: 3.1, 3.2
Rozdział 4.: 4.1.1, 4.1.2, 4.2.1, 4.2.2
Podsumowanie

Spis treści

Wprowadzenie **4**

1 Podstawy teoretyczne w kontekście uczenia operatorów **9**
1.1 Przestrzenie funkcji i operatory 9
1.2 Teoretyczne gwarancje skuteczności modeli predykcyjnych 12
1.3 Reprezentacja danych 14

2 Klasyfikacja operatorów matematycznych **16**
2.1 Operatory liniowe i nieliniowe 16
2.2 Operatory lokalne i nielokalne 17
2.3 Operatory rozwiązań równań różniczkowych 18

3 Architektury sieci operatorowych **19**
3.1 DeepONet (Deep Operator Network) 19
3.2 Operator Neuronowy Fouriera (FNO) 22

4 Eksperymenty **25**
4.1 Analizowane problemy analityczne 25
4.1.1 Model wzrostu logistycznego 26
4.1.2 Model wzrostu RCK 26
4.1.3 Równanie Burgersa 27
4.2 Przedstawienie otrzymanych wyników 27
4.2.1 Model wzrostu logistycznego 28
4.2.2 Model wzrostu RCK 31
4.2.3 Równanie Burgersa 36

Podsumowanie **45**

Wprowadzenie

Wiele problemów występujących w naukach ścisłych, ekonomii oraz inżynierii wymaga rozwiązywania układów równań różniczkowych dla różnych wartości parametrów wejściowych oraz warunków początkowych. Modele tego typu stanowią fundament nowoczesnej analityki predykcyjnej, pojawiając się w mechanice molekularnej, dynamice płynów, ale również — co kluczowe w kontekście analizy Big Data – w modelowaniu złożonych procesów ekonomicznych, wycenie zaawansowanych instrumentów pochodnych czy analizie dynamicznych relacji popytu i podaży [1]. W praktyce wiele z tych problemów ma charakter silnie nieliniowy i wysokowymiarowy, przez co ich numeryczne rozwiązywanie jest ekstremalnie kosztowne obliczeniowo, tworząc wąskie gardło w procesach decyzyjnych. Klasyczne metody numeryczne, takie jak metody różnic skończonych, elementów skończonych czy metody spektralne, zazwyczaj wymagają bardzo gęstej dyskretyzacji przestrzeni i czasu. W warunkach biznesowych, gdzie optymalizacja portfela czy ocena ryzyka rynkowego wymaga tysięcy symulacji typu Monte Carlo, narzut obliczeniowy tych metod staje się barierą nie do zaakceptowania. Gwałtowny wzrost zapotrzebowania na moc obliczeniową, wraz ze wzrostem rozdzielczości modelu, ogranicza możliwość wykonywania analiz w czasie rzeczywistym, co w świecie finansów i logistyki przekłada się na realne straty operacyjne wynikające z opóźnień decyzyjnych.

W odpowiedzi na te wyzwania, w ostatnich latach coraz większe zainteresowanie wzbudzają metody uczenia maszynowego, pozwalające na efektywną aproksymację operatorów. Szczególną uwagę zwracają operatory neuronowe, których celem nie jest jedynie aproksymacja pojedynczej funkcji, lecz wyuczenie operatora mapującego przestrzeń parametrów wejściowych bezpośrednio na rozwiązanie równania [2]. Z punktu widzenia biznesowego, podejście to oferuje radykalną zmianę paradygmatu: kosztowny obliczeniowo proces uczenia modelu jest przenoszony do fazy przygotowawczej (ang. *offline*), natomiast faza wnioskowania (ang. *inference*) staje się niemal natychmiastowa. Pozwala to na wykonywanie w czasie rzeczywistym analiz scenariuszowych oraz optymalizację parametrów w skali, która dotychczas była nieosiągalna dla klasycznych solverów.

Celem niniejszej pracy jest analiza możliwości zastosowania operatorów neuronowych do aproksymacji rozwiązań równań różniczkowych oraz porównanie ich efektywności operacyjnej z klasyczną architekturą wielowarstwowego perceptronu (MLP, ang. *Multi–Layer Percep-*

tron). W związku z tak postawionym celem, główną hipotezą badawczą pracy jest założenie, że architektury operatorów neuronowych osiągają wyższą dokładność aproksymacji oraz lepszą zdolność generalizacji niż klasyczne sieci MLP przy problemach tej klasy. W szczególności praca skupia się na architekturach *DeepONet* [3] oraz *Fourier Neural Operator (FNO)* [4], które dzięki swojej zdolności do generalizacji operatorów w przestrzeniach funkcyjnych stanowią obiecujący kierunek rozwoju systemów wspomagania decyzji w środowiskach typu Big Data.

Motywacja i ograniczenia klasycznych architektur neuronowych

W matematyce operator jest szczególnym rodzajem odwzorowania działającego na elementach pewnej przestrzeni i przekształcającego je w elementy innej przestrzeni. W przeciwieństwie do klasycznych funkcji rzeczywistych, których argumentami i wartościami są najczęściej liczby, operator może działać na bardziej złożonych obiektach, takich jak funkcje, ciągi, macierze czy pola wektorowe, które w ogólności nazywa się wektorami[1].

W niniejszej pracy głównym obiektem zainteresowania są operatory działające na przestrzeniach funkcji. Oznacza to, że zarówno argumentem operatora, jak i jego wynikiem są funkcje. Formalnie pojęcie operatora można zdefiniować następująco:

Definicja 0.1 *Niech $\mathcal{A}$ oraz $\mathcal{U}$ będą przestrzeniami Banacha, zdefiniowanymi odpowiednio na dziedzinach $D \subset \mathbb{R}^d$ oraz $D' \subset \mathbb{R}^{d'}$. Operatorem $\mathcal{G}$ nazywa się odwzorowanie postaci*

$$\mathcal{G} : \mathcal{A} \to \mathcal{U}.$$

W praktyce oznacza to, że operator pobiera funkcję wejściową i przekształca ją w inną funkcję. Przykładem może być operator różniczkowy, który przypisuje funkcji jej pochodną, lub operator rozwiązujący równanie różniczkowe dla zadanych warunków początkowych.

Choć każdy operator jest funkcją w sensie matematycznym, pojęcie operatora stosuje się zwykle wtedy, gdy dziedzina lub przeciwdziedzina składa się z obiektów bardziej złożonych niż liczby rzeczywiste. Można więc powiedzieć, że operator jest uogólnieniem klasycznej funkcji na przestrzenie nieskończenie wymiarowe. W kontekście równań różniczkowych, operator często opisuje zależność między funkcją wejściową, np. warunkiem początkowym lub parametrami układu, a funkcją wyjściową, reprezentującą rozwiązanie problemu.

Współczesna analityka danych, szczególnie w obszarach modelowania procesów fizycznych i ekonomicznych, coraz częściej napotyka na problem odwzorowań zachodzących pomiędzy przestrzeniami funkcyjnymi. Podczas gdy klasyczne sieci typu MLP z powodzeniem rozwiązują problemy regresji i klasyfikacji na danych tabelarycznych, ich zdolność do generalizacji w zadaniach funkcyjnych jest istotnie ograniczona [1]. Kluczową słabością tych modeli jest ich

[1] Czyli wektor jest uogólnionym pojęciem obejmującym obiekty spełniające aksjomaty przestrzeni wektorowej.

sztywne powiązanie z konkretną, ustaloną dyskretyzacją danych wejściowych, co uniemożliwia ich efektywne skalowanie oraz pracę w środowiskach Big Data, gdzie struktura siatki może ulegać zmianom.

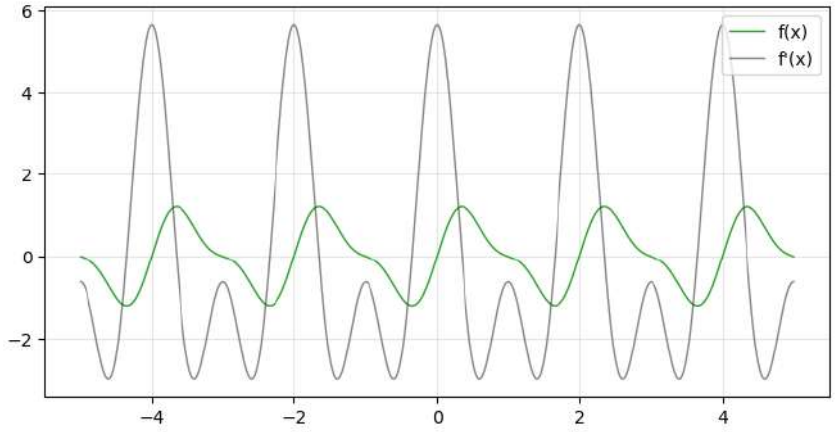
Eksperyment: Ograniczenia MLP w zadaniach pochodnej funkcyjnej

Aby zilustrować to ograniczenie, przeprowadzono eksperyment polegający na nauczeniu sieci neuronowej odwzorowania $f(x) \rightarrow f'(x)$. Rozważana rodzina funkcji została zdefiniowana jako:

$$f(x) = \sin(a\pi x) + b\sin(c\pi x) + d, \tag{1}$$

z analityczną postacią pochodnej:

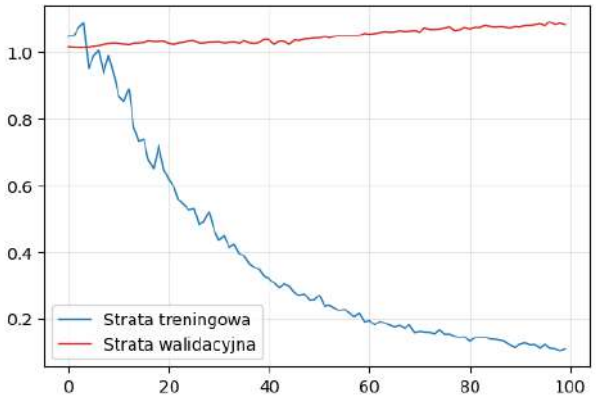
$$f'(x) = a\pi\cos(a\pi x) + bc\pi\cos(c\pi x). \tag{2}$$



Rysunek 1: Przykład funkcji z analizowanej rodziny. Parametry: $a = 1$, $b = 0.4$, $c = 2$, $d = 0$.

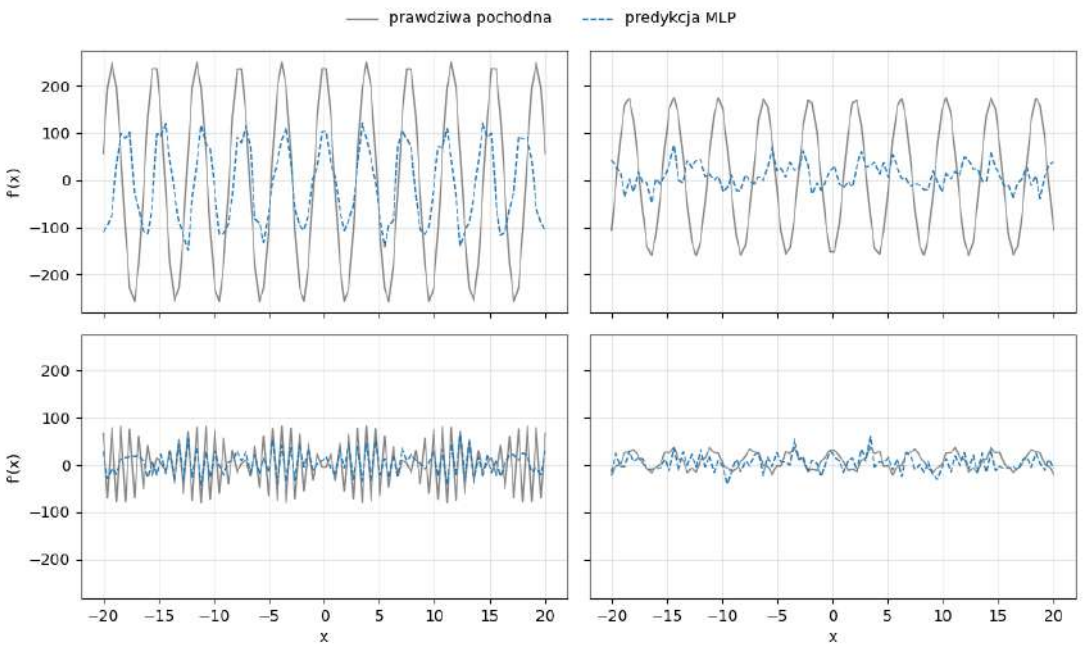
W badaniu zastosowano klasyczną architekturę MLP z trzema warstwami ukrytymi, po 128 neuronów każda. Model trenowano w oparciu o ustaloną siatkę punktów x. Choć sieć skutecznie minimalizowała błąd treningowy (MSE), wyniki na zbiorze walidacyjnym wskazały na brak zdolności do generalizacji.

Jak pokazano na Rysunku 2, błąd walidacyjny pozostawał w stagnacji, co potwierdza, że model nie wyuczył się rzeczywistego operatora różniczkowania, a jedynie przybliżonej interpolacji dla konkretnych punktów siatki.



Rysunek 2: Krzywe uczenia modelu MLP. Błąd treningowy systematycznie maleje, natomiast błąd walidacyjny pozostaje na zbliżonym poziomie, co wskazuje na przeuczenie w kontekście konkretnej siatki.

Dodatkowa analiza jakości predykcji na przykładowych próbkach (Rysunek 3) potwierdza ograniczoną zdolność modelu do generalizacji na wcześniej nieznanych funkcjach z tej samej rodziny.



Rysunek 3: Porównanie rzeczywistej oraz przewidywanej pochodnej dla wybranych próbek testowych.

Uzyskane wyniki wskazują, że mimo zdolności MLP do redukcji błędu treningowego, model nie jest w stanie poprawnie uogólnić zależności pomiędzy funkcjami a ich pochodnymi. Rozważana rodzina funkcji zawiera istotne składowe wysokoczęstotliwościowe oraz lokalne zmiany charakteru. W rezultacie klasyczne sieci typu MLP, które preferują globalnie gładkie aproksymacje funkcji, mają trudności z poprawnym odwzorowaniem takich zależności. W kon-

sekwencji uzasadnia to potrzebę stosowania metod uczących odwzorowań operatorowych, które są projektowane z myślą o pracy w przestrzeniach funkcji i lepiej wykorzystują ich strukturę funkcyjną.

W dalszej części pracy przedstawione zostaną rygorystyczne podstawy matematyczne, które pozwolą sformalizować pojęcie operatora oraz omówione zostanie, w jaki sposób wybrane architektury neuronowe adresują zidentyfikowane powyżej braki klasycznych rozwiązań.

Rozdział 1

Podstawy teoretyczne w kontekście uczenia operatorów

W niniejszym rozdziale przedstawiono pojęcia matematyczne, które stanowią teoretyczny fundament uczenia operatorów. W klasycznym uczeniu maszynowym dane przyjmują najczęściej postać wektorów o stałym wymiarze, np. ustrukturyzowanych zbiorów tabelarycznych. W zaawansowanej analityce Big Data coraz częściej analizuje się jednak sygnały ciągłe, takie jak szeregi czasowe o wysokiej częstotliwości, mapy przestrzenne czy dynamiczne procesy rynkowe. Zrozumienie przestrzeni, w których „żyją" te funkcje, oraz praw nimi rządzących jest niezbędne do poprawnego projektowania i trenowania operatorowych sieci neuronowych.

1.1. Przestrzenie funkcji i operatory

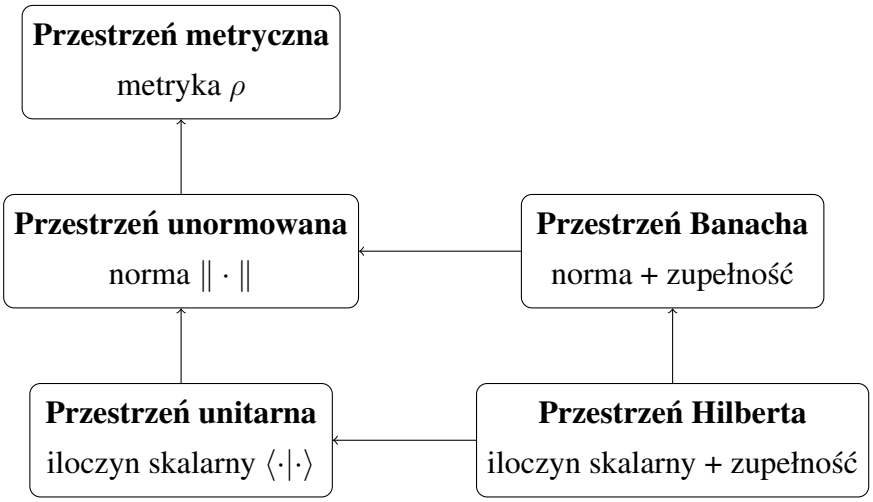
Aby algorytmy mogły uczyć się na całych funkcjach – zamiast na punktowych próbkach – konieczne jest zdefiniowanie odpowiedniego środowiska matematycznego. W analizie operatorowej odwzorowania działają na przestrzeniach funkcyjnych, co wymaga przejścia z klasycznej geometrii euklidesowej do bardziej ogólnych struktur.

Przestrzenie Banacha i Hilberta jako środowisko danych

W klasycznej analityce odległość między wektorami danych mierzy się najczęściej metryką euklidesową. W przypadku ciągłych strumieni danych wprowadza się pojęcie przestrzeni unormowanych. Metryka jest w nich indukowana przez normę, która mierzy „wielkość" sygnału.

Jeśli przestrzeń unormowana jest zupełna (co gwarantuje, że procesy graniczne, takie jak zbieżność algorytmów optymalizacyjnych, nie „wypadają" poza tę przestrzeń), nazywamy ją **przestrzenią Banacha** [5, s. 3–4]. Gdy dodatkowo zdefiniowany jest w niej iloczyn skalarny (pozwalający m.in. na mierzenie kątów i rzutowanie ortogonalne), przestrzeń ta staje się **przestrzenią Hilberta** [6, Def., s. 88]. Z perspektywy sieci neuronowych to właśnie w przestrze-

niach Banacha i Hilberta definiowane są funkcje wejściowe (np. warunki początkowe symulacji) oraz wyjściowe (wynikowe predykcje).



Rysunek 1.1: Hierarchia przestrzeni funkcyjnych wykorzystywanych do reprezentacji ciągłych danych wejściowych.

Regularność danych i przestrzenie Sobolewa

Dane z rzeczywistych procesów (np. finansowych czy fizycznych) są często zaszumione i wykazują gwałtowne zmiany. Właściwość ta w matematyce określana jest mianem **regularności**. W kontekście uczenia maszynowego regularność funkcji wejściowych determinuje zdolność modelu do zbieżności i uniknięcia przeuczenia (ang. *overfitting*) [7].

Aby mierzyć tę własność, wprowadza się przestrzenie całkowalne w sensie Lebesgue'a L^p [8, Def. 3.5]. Dla $p \in [1, \infty)$ definiuje się je jako:

$$L^p(\Omega) \coloneqq \left\{ f : \int_\Omega |f(\mathbf{x})|^p \, d\mathbf{x} < \infty \right\}, \tag{1.1}$$

natomiast dla $p = \infty$:

$$L^\infty(\Omega) \coloneqq \left\{ f : \operatorname*{ess\,sup}_{\mathbf{x}\in\Omega} |f(\mathbf{x})| < \infty \right\}. \tag{1.2}$$

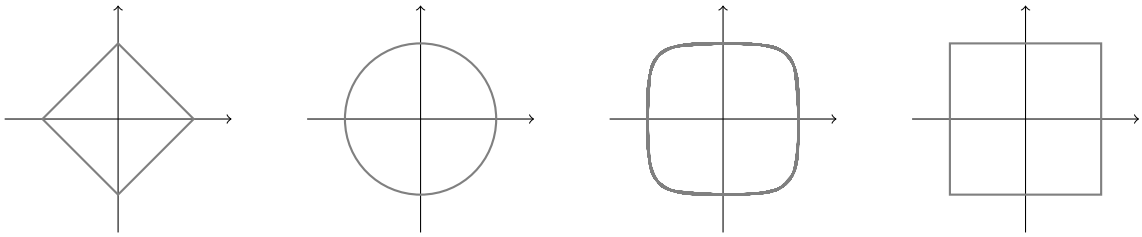
Geometria tych przestrzeni zmienia się w zależności od parametru p (Rysunek 1.2), co wpływa na zachowanie funkcji kosztu podczas treningu sieci.

Na bazie przestrzeni L^p definiuje się przestrzeń Sobolewa, która uwzględnia nie tylko same wartości sygnału, ale również dynamikę jego zmian, czyli pochodnych.

Definicja 1.1 (Przestrzeń Sobolewa) *[8, Def. 3.20] Niech $k, n \in \mathbb{N} \cup \{0\}$ oraz $p \in [1, \infty]$, a Ω będzie otwartym podzbiorem $\mathbb{R}^n$. Przestrzeń Sobolewa $W^{k,p}(\Omega)$ to zbiór:*

$$W^{k,p}(\Omega) \coloneqq \{u \in L^p(\Omega) : D^\alpha u \in L^p(\Omega) \; \forall \alpha \in \mathbb{N}^n, |\alpha| \leq k\}, \tag{1.3}$$

gdzie $D^\alpha u$ oznacza słabą pochodną [8, Def. 3.10] funkcji u rzędu α.



Rysunek 1.2: Kula jednostkowa w różnych normach L^p na $\mathbb{R}^2$, ilustrująca jak różne metryki karzą odchylenia błędów predykcji.

Jeżeli funkcja uczy model na sygnałach, które nie należą do odpowiedniej przestrzeni Sobolewa (np. posiadają niemożliwe do zróżniczkowania skoki), tradycyjne metody analityczne i sieci płytkie mogą ulec destabilizacji, co zmusza do korzystania z operatorów wyższego rzędu.

Pochodna Frécheta a wsteczna propagacja błędu

Trening każdej współczesnej sieci neuronowej opiera się na algorytmie wstecznej propagacji błędu, który wykorzystuje pochodną funkcji straty do aktualizacji wag. Kiedy jednak wejściem do sieci i jej wyjściem są funkcje, klasyczny rachunek różniczkowy okazuje się niewystarczający. Konieczne jest użycie pochodnej Frécheta, która uogólnia koncepcję gradientu na przestrzenie Banacha. Z matematycznego i obliczeniowego punktu widzenia pozwala ona na znalezienie najlepszego liniowego przybliżenia operatora nieliniowego wokół danego punktu.

Definicja 1.2 (Pochodna Frécheta) *[9, Def. 3.2] Niech X i Y będą przestrzeniami Banacha, a $U \subset X$ zbiorem otwartym. Funkcja $f : U \to Y$ jest **różniczkowalna w sensie Frécheta** w punkcie $x \in U$, jeżeli istnieje operator liniowy i ograniczony $Df(x) \in \mathcal{L}(X, Y)$ taki, że:*

$$\lim_{\|h\|\to 0} \frac{\|f(x+h) - f(x) - Df(x)[h]\|}{\|h\|} = 0. \tag{1.4}$$

*Operator $Df(x)$ nazywa się **pochodną Frécheta** funkcji f w punkcie x.*

Własność ta, w połączeniu ze zwartością operatorów [10, Def. 11.2], która gwarantuje ograniczoność przekształceń, zapewnia, że procesy optymalizacyjne w modelach operatorowych są w stanie zbiegać do optymalnych parametrów podczas uczenia na dużych zbiorach danych.

1.2. Teoretyczne gwarancje skuteczności modeli predykcyjnych

Z biznesowego i inżynierskiego punktu widzenia, przed wdrożeniem kosztownego procesu budowy modelu analitycznego kluczowe jest upewnienie się, że docelowa architektura posiada wystarczającą pojemność informacyjną (ang. *capacity*), by nauczyć się pożądanych zależności. Dowodów tych dostarczają twierdzenia o uniwersalnej aproksymacji.

Twierdzenie o uniwersalnej aproksymacji

Podstawowe twierdzenie o uniwersalnej aproksymacji (UAT, ang. *Universal Approximation Theorem*) dowodzi, że odpowiednio duża klasyczna sieć neuronowa może przybliżyć z dowolną precyzją każdą funkcję ciągłą [11, 12]. Zostało ono jednak sformułowane dla skończenie wymiarowych wektorów.

Twierdzenie 1.3 (Twierdzenie o uniwersalnej aproksymacji) *Niech $K \subset \mathbb{R}^n$ będzie zbiorem zwartym, a $C(K)$ oznacza przestrzeń funkcji ciągłych na K. Wówczas dla każdej funkcji $f \in C(K)$ i każdego $\epsilon > 0$ istnieje sieć neuronowa $\hat{f}$ taka, że:*

$$\sup_{x \in K} |f(x) - \hat{f}(x)| < \epsilon. \tag{1.5}$$

Ponieważ sieci operatorowe rozwiązują problemy w znacznie bardziej złożonych przestrzeniach funkcyjnych, konieczne było sformułowanie uogólnienia tego twierdzenia. Gwarantuje ono, że model neuronowy jest w stanie z sukcesem aproksymować nie tylko punkty danych, ale całe nieliniowe operatory (np. procesy fizyczne lub ewolucje giełdowe).

Twierdzenie 1.4 (Twierdzenie o uniwersalnej aproksymacji dla operatorów) *[13, Tw. 1] Niech σ będzie nieliniową, ciągłą funkcją aktywacji. Niech X będzie przestrzenią Banacha, a zbiory $K_1 \subset X$, $K_2 \subset \mathbb{R}^d$ oraz $V \subset C(K_1)$ będą zwarte. Niech $G : V \to C(K_2)$ będzie operatorem ciągłym.*

Wówczas dla każdego $\varepsilon > 0$ istnieje operator neuronowy G_θ o skończonej liczbie parametrów taki, że:

$$\sup_{u \in V} \|G(u) - G_\theta(u)\|_{C(K_2)} < \varepsilon. \tag{1.6}$$

Należy jednak pamiętać, że UAT gwarantuje jedynie istnienie optymalnych wag. Rzeczywisty sukces modelu zależy od algorytmu optymalizacyjnego i zdolności do generalizacji, czyli uniknięcia błędu uogólnienia.

Klątwa wymiarowości i twierdzenie Kołmogorowa

Jednym z najpoważniejszych problemów w analizie Big Data jest tzw. klątwa wymiarowości (ang. *curse of dimensionality*). Wraz ze wzrostem liczby zmiennych wejściowych (wymiarów problemu), złożoność obliczeniowa i zapotrzebowanie na dane rosną w sposób wykładniczy, co często czyni klasyczne metody numeryczne bezużytecznymi w środowisku produkcyjnym.

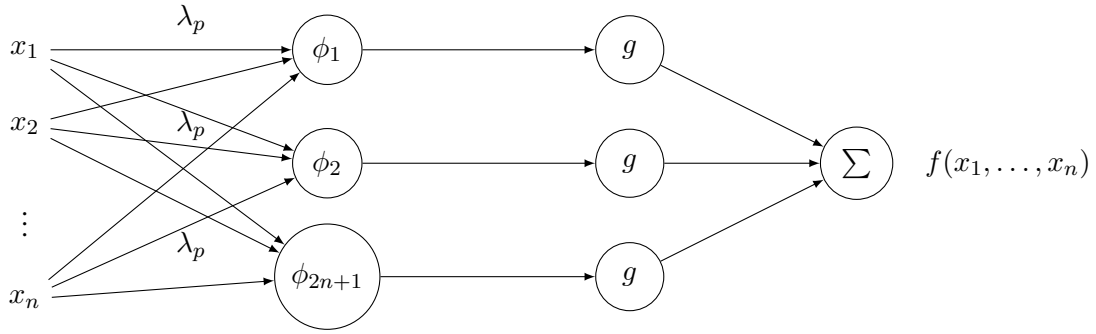
Rozwiązania tego problemu dla sieci neuronowych dostarcza Twierdzenie o superpozycji Kołmogorowa [14, 15].

Twierdzenie 1.5 (Twierdzenie o superpozycji Kołmogorowa) *Każda ciągła funkcja n zmiennych $f(x_1, \ldots, x_n)$ określona w obszarze $[0,1]^n$ może być przedstawiona jako:*

$$f(x_1, \ldots, x_n) = \sum_{q=1}^{2n+1} g\left(\sum_{p=1}^{n} \lambda_p \phi_q(x_p)\right), \tag{1.7}$$

gdzie $\lambda_p \in (0,1]$ są stałymi, ϕ_q są jednowymiarowymi, ciągłymi funkcjami rosnącymi, a g jest funkcją ciągłą zależną od f.

Powyższe twierdzenie oznacza w praktyce biznesowej, że nawet najbardziej skomplikowane procesy wielowymiarowe można w pełni opisać jako kombinację prostych funkcji jednowymiarowych (Rysunek 1.3). To właśnie ta własność dekompozycji sprawia, że sieci neuronowe mogą omijać klątwę wymiarowości, stanowiąc klucz do tworzenia efektywnych obliczeniowo architektur predykcyjnych w projektach klasy Big Data.



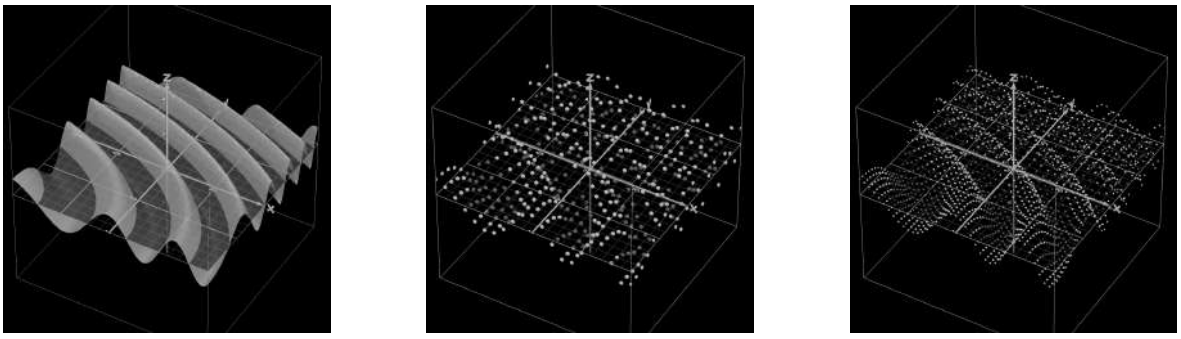
Rysunek 1.3: Interpretacja twierdzenia Kołmogorowa jako jednokierunkowej sieci neuronowej, ilustrująca redukcję złożoności poprzez dekompozycję wymiarów.

1.3. Reprezentacja danych

Jednym z najważniejszych zagadnień w analizie Big Data jest sposób reprezentacji informacji. W praktycznych zastosowaniach dane pochodzące z systemów biznesowych, procesów przemysłowych, rynków finansowych czy symulacji naukowych mogą przyjmować bardzo różne formy: od pojedynczych obserwacji, przez szeregi czasowe, aż po złożone obiekty przestrzenno–czasowe. Sposób ich reprezentacji wpływa nie tylko na możliwość wykorzystania określonych metod uczenia maszynowego, lecz również na efektywność przechowywania danych, koszt ich przetwarzania oraz zdolność modeli do uogólniania wiedzy na nowe przypadki.

Problem ten staje się szczególnie istotny w przypadku operatorów neuronowych. W odróżnieniu od klasycznych modeli uczenia maszynowego, które uczą się zależności pomiędzy skończonymi wektorami cech, operator neuronowy ma za zadanie odwzorowywać funkcje na funkcje. Oznacza to konieczność przejścia pomiędzy światem obiektów ciągłych a ich reprezentacją cyfrową. Dobór odpowiedniej reprezentacji danych wpływa zatem bezpośrednio na jakość aproksymacji operatora, skalowalność rozwiązania oraz możliwość pracy z danymi o różnej rozdzielczości.

Spośród różnych metod reprezentacji funkcji, podczas implementacji wykorzystywana zostanie **reprezentacja siatkowa** (ang. *grid–based*). Jej ideą jest dyskretyzacja dziedziny funkcji, czyli zamiast reprezentować ją na całym ciągłym obszarze Ω, rozpatruje się jej wartości jedynie w skończonej liczbie punktów tworzących siatkę obliczeniową $\{\mathbf{x}_{i_1,\ldots,i_n}\}_{i_k=0}^{m_k} \subset \Omega$.



(a) (b) (c)

Rysunek 1.4: Dyskretyzacja funkcji ciągłej (a) na siatkach o niższej (b) i wyższej (c) rozdzielczości.

Każdy punkt siatki można zdefiniować następująco:

$$\mathbf{x}_{i_1,\ldots,i_n} = (x_1^{(i_1)}, \ldots, x_n^{(i_n)}) \tag{1.8}$$

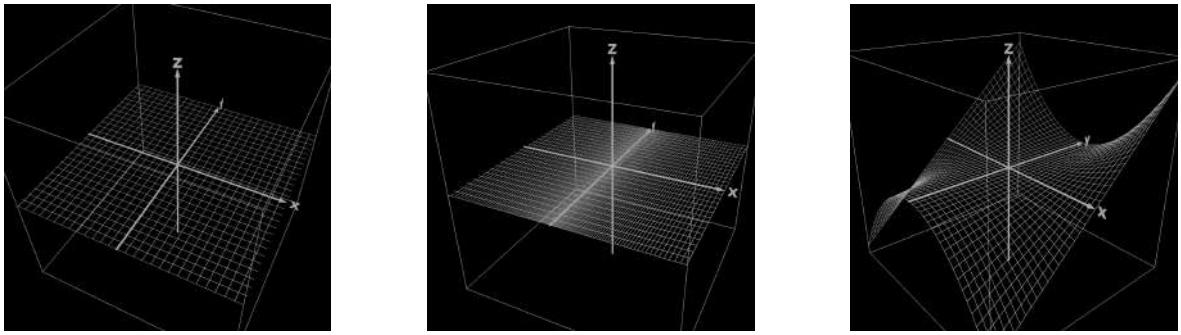
gdzie n to liczba wymiarów przestrzeni dziedziny Ω. Wówczas dyskretna reprezentacja funkcji przyjmuje postać:

$$\hat{f}(x) = (f(\mathbf{x}_{0_0,\ldots,0_n}), \ldots, f(\mathbf{x}_{m_0,\ldots,m_n})), \tag{1.9}$$

gdzie m to liczba punktów tworzących siatkę obliczeniową.

Z perspektywy analizy Big Data problem ten przypomina klasyczny kompromis pomiędzy ilością przechowywanych informacji a kosztem ich przetwarzania. Gęstsza siatka dostarcza więcej danych opisujących badane zjawisko, jednak zwiększa wymagania pamięciowe oraz czas obliczeń. W dużych zbiorach danych różnice te mogą być bardzo istotne, dlatego w praktyce konieczne jest znalezienie równowagi pomiędzy dokładnością reprezentacji a efektywnością obliczeniową.

Najprostszym typem siatki jest **siatka regularna** (ang. *uniform grid*), dla której odstępy siatki na jednym wymiarze są stałe, ale w obrębie różnych wymiarów mogą być inne, czyli każdy punkt spełnia $x_k^{(i_k)} = i_k h_k$. Oprócz niej, jest też siatka prostoliniowa (ang. *rectilinear grid*), w której kroki siatki w obrębie jednego wymiaru nie muszą być zachowane oraz siatka krzywoliniowa (ang. *curvilinear grid*), dla której komórki nie są wyrównane osiowo z euklidesowym układem współrzędnych, czyli mogą być krzywymi.



(a) (b) (c)

Rysunek 1.5: Przykłady strukturyzowanych siatek: regularna (a), prostoliniowa (b) i krzywoliniowa (c).

Reprezentacja siatkowa odgrywa szczególnie istotną rolę w operatorach neuronowych, ponieważ większość współczesnych architektur operuje bezpośrednio na dyskretnych próbkach funkcji. W klasycznych sieciach neuronowych zmiana rozdzielczości siatki zwykle wymaga zmiany architektury lub ponownego uczenia modelu. Jednym z głównych celów operatorów neuronowych jest natomiast uzyskanie **niezmienniczości względem dyskretyzacji** (ang. *resolution invariance*). Z własnością tą związane jest również zjawisko *resolution transfer*, polegające na możliwości trenowania modelu na siatkach o niższej rozdzielczości, a następnie stosowania go na siatkach gęstszych. Dzięki temu operator neuronowy może aproksymować operator ciągły, a nie jedynie konkretne odwzorowanie między ustalonymi wektorami próbek.

Rozdział 2

Klasyfikacja operatorów matematycznych

W uczeniu sieci neuronowych kluczowe jest dopasowanie architektury do charakterystyki rozwiązywanego problemu. Przykładowo, architektura **Fourier Neural Operator** (FNO) wykazuje wysoką skuteczność w modelowaniu operatorów gładkich i ciągłych, podczas gdy jej efektywność maleje w przypadku występowania nieciągłości lub ostrych gradientów (np. w modelowaniu fal uderzeniowych). W niniejszym rozdziale opisano kryteria klasyfikacji operatorów, które determinują wybór optymalnej strategii modelowania.

2.1. Operatory liniowe i nieliniowe

Operatory liniowe

Operator liniowy T definiuje się poprzez spełnienie zasad addytywności oraz jednorodności. Formalnie wymagane są następujące założenia [16, s. 82–83]:

- dziedzina $\mathcal{D}(T)$ musi być przestrzenią wektorową,
- zbiór wartości $\mathcal{R}(T)$ musi leżeć w przestrzeni wektorowej nad tym samym ciałem (zazwyczaj $\mathbb{R}$ lub $\mathbb{C}$).

Dla dowolnych funkcji $f, g \in \mathcal{D}(T)$ oraz skalarów α, β, operator liniowy spełnia warunek:

$$T(\alpha f + \beta g) = \alpha T(f) + \beta T(g). \tag{2.1}$$

Z perspektywy uczenia operatorów liniowość niesie istotne zalety:

1. **Niezmienniczość względem przesunięcia:** Pozwala na diagonalizację operatora za pomocą transformacji Fouriera, co stanowi teoretyczne uzasadnienie dla architektury FNO.
2. **Istnienie wartości i funkcji własnych:** Możliwość przedstawienia operatora jako macierzy diagonalnej w przestrzeni spektralnej, gdzie model uczy się wyłącznie wag wzmacniających poszczególne częstotliwości.
3. **Zasada superpozycji:** Umożliwia rekonstrukcję odpowiedzi na złożony sygnał jako sumy odpowiedzi na sygnały proste.

W przestrzeniach unormowanych dla operatora liniowego ciągłość jest równoważna ograniczoności. Operator T jest ograniczony, gdy istnieje stała $M > 0$ taka, że dla każdego $x \in \mathcal{D}(T)$ zachodzi:

$$\|T(x)\| \leq M\|x\|. \tag{2.2}$$

W praktyce (np. w modelowaniu pogody) własność ta gwarantuje stabilność systemu – drobny szum w danych wejściowych skutkuje jedynie proporcjonalnie niewielkim błędem na wyjściu.

Operatory nieliniowe

Większość procesów rzeczywistych wykazuje nieliniowość, co oznacza złamanie zasady superpozycji. Operator nieliniowy N wprowadza autooddziaływanie, w którym ewolucja systemu zależy od jego stanu w sposób nieproporcjonalny. W takich sytuacjach metody numeryczne wymagają iteracyjnej linearyzacji, np. za pomocą pochodnej Frécheta, co jest kosztowne obliczeniowo. Operatorowe sieci neuronowe oferują w tym zakresie przewagę, ucząc się bezpośredniego odwzorowania. Architektura sieci jest zazwyczaj złożeniem liniowych operatorów całkowych K oraz nieliniowych funkcji aktywacji σ:

$$N(x) = \sigma(K(x) + b). \tag{2.3}$$

Funkcje aktywacji wprowadzają sprzężenie modów (ang. *mode coupling*), umożliwiając modelowanie złożonych zjawisk, takich jak kaskady energii, przy jednoczesnej optymalizacji procesu uczenia poprzez eliminację słabych składowych sygnału.

2.2. Operatory lokalne i nielokalne

Operatory lokalne

Operator lokalny N rzędu k zależy w każdym punkcie x jedynie od wartości funkcji f oraz jej pochodnych do rzędu k:

$$N(f)(x) = F\left(x, f(x), Df(x), \ldots, D^k f(x)\right). \tag{2.4}$$

Przykładem liniowym jest dyfuzja ciepła, natomiast nieliniowym jest m.in. równanie Burgersa, w którym wartość funkcji jest mnożona przez jej gradient.

Operatory globalne

Operator nielokalny (całkowy) charakteryzuje się tym, że wartość $(Tf)(x)$ zależy od wartości funkcji f w całej domenie D:

$$(Tf)(x) = \int_D k(x, y) f(y)\, dy, \tag{2.5}$$

gdzie $k(x, y)$ jest jądrem operatora. W rozwiązywaniu równań różniczkowych $\mathcal{L}u = f$, rozwiązanie często zapisuje się przy użyciu funkcji Greena $G(x, y)$ [7, s. 33–40]:

$$u(x) = \int_D G(x, y) f(y)\, dy. \tag{2.6}$$

Dzięki tej własności operatory neuronowe, zamiast dopasowywać się do lokalnego szumu, uczą się uśredniania wartości z całej domeny, co czyni je znacznie bardziej stabilnymi niż metody bezpośrednio przybliżające pochodne.

2.3. Operatory rozwiązań równań różniczkowych

Równanie różniczkowe to matematyczna zależność wiążąca funkcję z jej pochodnymi. Można je podzielić na **zwyczajne** (ODE, ang. *Ordinary Differential Equations*) – gdzie poszukiwana funkcja zależy od jednej zmiennej (np. czasu t) – oraz **cząstkowe** (PDE, ang. *Partial Differential Equations*), gdzie zależność dotyczy wielu zmiennych, np. czas t i współrzędne przestrzenne x, y, z. Ogólne równanie dla rozwiązania $u(x)$ na domenie $\Omega \subset \mathbb{R}^d$ zapisuje się jako:

$$\mathcal{L}(u, a) = 0, \quad x \in \Omega, \tag{2.7}$$

gdzie $\mathcal{L}$ jest operatorem różniczkowym, a $a(x)$ parametrami wejściowymi układu.

Koncepcja **operatora rozwiązania** $\mathcal{G}$ polega na odwzorowaniu całej przestrzeni parametrów wejściowych $\mathcal{A}$ w przestrzeń rozwiązań $\mathcal{U}$:

$$\mathcal{G} : \mathcal{A} \to \mathcal{U}, \quad \mathcal{G}(a) = u. \tag{2.8}$$

Jest to odwrotność operatora różniczkowego ($\mathcal{G} = \mathcal{L}^{-1}$). Podczas gdy operator $\mathcal{L}$ działa lokalnie poprzez pochodne, operator rozwiązania $\mathcal{G}$ jest operatorem globalnym, co stanowi podstawę efektywności operatorowych sieci neuronowych w systemach predykcyjnych.

Rozdział 3

Architektury sieci operatorowych

W niniejszym rozdziale omówione zostaną kluczowe architektury sieci neuronowych wykorzystywane do aproksymacji operatorów. Głównym celem tych modeli jest przetwarzanie ciągłych przestrzeni funkcyjnych. Klasyczne modele uczenia maszynowego, takie jak wielowarstwowy perceptron czy standardowe sieci konwolucyjne, są z natury dostosowane do danych wejściowych o stałym wymiarze i napotykają istotne ograniczenia w zadaniach, gdzie wymiarowość lub gęstość danych może się zmieniać. Zamiast uczyć się lokalnego mapowania typu punkt–punkt, architektury operatorowe uogólniają wiedzę o systemach dynamicznych i uczą się relacji pomiędzy całymi rodzinami funkcji.

3.1. DeepONet (Deep Operator Network)

Architektura DeepONet stanowi przełom w praktycznym wykorzystaniu twierdzenia o uniwersalnej aproksymacji dla operatorów [13]. Jej innowacyjność oraz unikalność wynika z podziału na dwie współpracujące podsieci, z których jedna zajmuje się analizą funkcji wejściowej, a druga ciągłą reprezentacją dziedziny. Podsieci te nazywane są odpowiednio **siecią gałęzi** (ang. *Branch Network*) oraz **siecią pnia** (ang. *Trunk Network*).

Sieć gałęzi (Branch Network)

Zadaniem sieci gałęzi jest ekstrakcja istotnych cech z funkcji wejściowej u poprzez przekształcenie jej w wektor ukrytych współczynników. W kontekście analizy danych, sieć ta uczy się, jak silny wpływ na globalny charakter procesu ma konkretny profil wejściowy, np. początkowy układ temperatur lub historyczna krzywa stóp procentowych.

W podstawowej architekturze DeepONet funkcja wejściowa musi zostać zdyskretyzowana w zdefiniowanych punktach pomiarowych, nazywanych sensorami. Wejście do sieci przyjmuje

wówczas następującą postać:

$$u = [u(x_1), u(x_2), \ldots, u(x_m)]^T, \tag{3.1}$$

gdzie u jest wektorem wartości funkcji obliczonych w m stałych punktach kontrolnych (np. odczytach z czujników IoT).

Jeżeli zastosowana zostanie architektura wielowarstwowego perceptronu (MLP), wyjściem z sieci gałęzi będzie wektor współczynników b:

$$b = (b_1, b_2, \ldots, b_p)^T = \text{Branch}(u), \tag{3.2}$$

gdzie p oznacza liczbę funkcji bazowych wygenerowanych przez sieć pnia. Intuicyjnie rzecz ujmując, sieć gałęzi agreguje i streszcza wielowymiarową informację zawartą w sygnale wejściowym u do postaci p kluczowych cech (współczynników).

Warto zaznaczyć, że w przypadku klasycznego perceptronu punkty sensorowe muszą być z góry ustalone i stałe zarówno dla fazy uczenia, jak i inferencji. Jeżeli jednak zastosowana zostanie bardziej zaawansowana architektura – na przykład sieci konwolucyjne z mechanizmem *global pooling* lub grafowe sieci neuronowe – model zyskuje elastyczność i jest w stanie przyjmować na wejściu sygnały o zmiennej rozdzielczości, co jest kluczowe w skalowalnych rozwiązaniach Big Data.

Sieć pnia (Trunk Network)

Drugim filarem architektury DeepONet jest sieć pnia. Podczas gdy sieć gałęzi analizuje warunki wejściowe problemu, sieć pnia uczy się ciągłej reprezentacji dziedziny wyjściowej. Sieć ta przyjmuje na wejściu współrzędne punktu y (np. konkretny czas w przyszłości lub wybraną lokalizację przestrzenną), w których predykcje operatora bedą wyznaczone.

Zadaniem sieci pnia jest wygenerowanie wektora p funkcji bazowych dla ewaluowanego punktu:

$$t(y) = (\tau_1(y), \tau_2(y), \ldots, \tau_p(y))^T = \text{Trunk}(y). \tag{3.3}$$

Zamiast narzucać modelowi z góry określony zestaw klasycznych funkcji bazowych, jak ma to miejsce w tradycyjnych metodach spektralnych, sieć pnia w sposób adaptacyjny sama uczy się optymalnej bazy, która najlepiej opisuje przestrzeń rozwiązań analizowanego problemu.

Ogromną przewagą biznesową i operacyjną sieci pnia jest jej niezmienniczość względem siatki (ang. *mesh invariance*). Ponieważ sieć aproksymuje ciągłą funkcję współrzędnych y, nie jest ona ograniczona do konkretnej dyskretyzacji przestrzeni. W praktyce oznacza to możliwość wytrenowania modelu na rzadkiej i taniej w akwizycji siatce punktów, a następnie odpytywania modelu w fazie inferencji o predykcje w dowolnym innym punkcie z nieskończoną rozdzielczością. Tego rodzaju elastyczność pozwala na drastyczną redukcję kosztów obliczeniowych w porównaniu ze standardowymi modelami sieci neuronowych.

Łączenie obu sieci

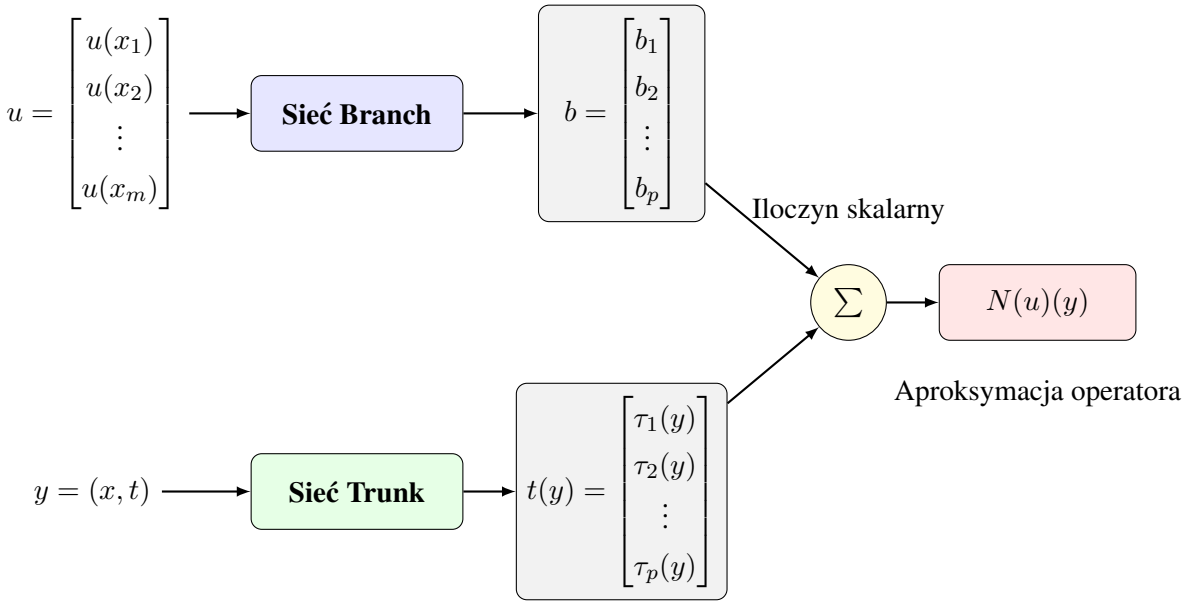
Ostatnim krokiem działania architektury DeepONet jest złączenie wektora współczynników z wektorem funkcji bazowych za pomocą iloczynu skalarnego:

$$N(u)(y) = \sum_{k=1}^{p} b_k \tau_k(y) + b_0, \tag{3.4}$$

gdzie:

- b_k to k-ty współczynnik wektora wygenerowanego przez sieć *branch*, zależny wyłącznie od funkcji wejściowej u,
- $\tau_k(y)$ to k-ta funkcja bazowa wygenerowana przez sieć *trunk*, zależna wyłącznie od współrzędnych punktu y,
- b_0 to wyraz wolny (ang. *bias*) ułatwiający dopasowanie modelu do danych.

Powyższy iloczyn skalarny stanowi istotę działania architektury. DeepONet *de facto* uczy się adaptacyjnej bazy funkcji. Zamiast korzystać ze sztywnych baz, jak w klasycznych metodach numerycznych, sieć *trunk* w sposób dynamiczny buduje optymalną bazę dla danego problemu, a sieć *branch* wylicza dla niej odpowiednie współczynniki. Dzięki temu model zachowuje elastyczność i jest w stanie dostosować postać rozwiązania do wysoce nieliniowych wymuszeń, które w klasycznych solverach wymagałyby stosowania bardzo gęstej siatki obliczeniowej.



Rysunek 3.1: Schemat architektury DeepONet. Sieć *Branch* przetwarza zdyskretyzowaną funkcję wejściową, generując współczynniki b_k, podczas gdy sieć *Trunk* buduje funkcje bazowe $\tau_k(y)$ zależne od współrzędnych dziedziny.

3.2. Operator Neuronowy Fouriera (FNO)

Podstawą architektury operatora FNO [17] jest twierdzenie o splocie, które zakłada, że splot w przestrzeni fizycznej jest równoważny mnożeniu punktowemu w przestrzeni spektralnej. Dzięki temu możliwe jest zastąpienie kosztownej operacji całkowej operatora nielokalnego:

$$(Lu)(x) = \int_D k(x, y)u(y)\, dy \tag{3.5}$$

operacją w dziedzinie częstotliwości (domena Fouriera):

$$(\mathcal{F}Lu)(x) = R \cdot (\mathcal{F}u)(x), \tag{3.6}$$

gdzie $\mathcal{F}$ oznacza transformację Fouriera, a R jest macierzą wag uczonych przez sieć. Z perspektywy architektury oznacza to, że model uczy się adaptacyjnego filtra nakładanego na częstotliwości bazowe sygnału. Wartościami w macierzy R są liczby zespolone, co pozwala sieci na jednoczesne uczenie się amplitudy oraz przesunięcia fazowego poszczególnych komponentów częstotliwościowych.

Przewagi parametryzacji spektralnej

Parametryzacja w przestrzeni częstotliwości niesie za sobą istotne korzyści w porównaniu ze splotem w przestrzeni rzeczywistej:

- **Globalny zasięg** (ang. *Global Receptive Field*): W przeciwieństwie do klasycznych splotów (np. w sieciach CNN), gdzie jądra są małych wymiarów (np. 4×4) i uczą się zależności lokalnych, macierz R oddziałuje na strukturę globalną sygnału. Każda wartość w przestrzeni spektralnej zależy od wszystkich punktów w przestrzeni fizycznej, co jest kluczowe w modelowaniu systemów fizycznych, gdzie perturbacja w jednym punkcie wpływa na cały układ.
- **Efektywność obliczeniowa:** Zastosowanie szybkiej transformacji Fouriera (FFT, ang. *Fast Fourier Transform*) redukuje złożoność obliczeniową operacji z $\mathcal{O}(n^2)$ do $\mathcal{O}(n \log n)$, gdzie n oznacza liczbę punktów dyskretyzacji. Jest to kluczowe dla skalowalności systemów klasy Big Data.

Regularyzacja i kompresja przez ograniczenie modów

Kluczowym hiperparametrem w architekturze FNO jest liczba przetwarzanych częstotliwości, nazywanych modami. Ograniczenie liczby modów wprowadza do modelu:

- **Regularyzację:** Odfiltrowanie wysokich częstotliwości wymusza gładkość rozwiązania, co zwiększa odporność modelu na szum pomiarowy.

– **Redukcję wymiarowości:** Każda „odcięta” moda zmniejsza liczbę parametrów macierzy wag R, co przyspiesza uczenie i ogranicza zapotrzebowanie na pamięć operacyjną, co jest istotne przy pracy z bardzo dużymi zbiorami danych.

W praktyce FNO oferuje unikalną kombinację elastyczności operatorowej z wysoką wydajnością algorytmiczną, co czyni go jednym z najefektywniejszych narzędzi w rozwiązywaniu sparametryzowanych równań różniczkowych cząstkowych oraz modelowaniu złożonych, dynamicznych procesów ekonomicznych.

Rola nieliniowości w FNO

Sama macierz R w przestrzeni spektralnej, mimo swojej skuteczności w wykrywaniu globalnych zależności, stanowi operację ściśle liniową. Aby architektura mogła stać się uniwersalnym aproksymatorem operatorów nieliniowych, w każdej warstwie modelu wprowadzana jest nieliniowa funkcja aktywacji (np. ReLU, GELU lub SiLU) oraz dodatkowa transformacja liniowa.

Pojedynczą warstwę modelu FNO definiuje się następująco:

$$v_{l+1}(x) = \sigma\left((\mathcal{F}^{-1}(R_l \cdot \mathcal{F}v_l))(x) + Wv_l(x)\right), \tag{3.7}$$

gdzie:

– σ oznacza nieliniową funkcję aktywacji,
– $\mathcal{F}^{-1}(R_l \cdot \mathcal{F}v_l)$ stanowi opisaną wcześniej ścieżkę spektralną (transformacja do dziedziny częstotliwości, filtracja macierzą wag R_l i powrót do przestrzeni fizycznej),
– $Wv_l(x)$ to dodatkowa transformacja liniowa (często splotowa), która zapewnia lokalną stabilność i wzbogaca reprezentację sygnału.

Dzięki tak zaprojektowanej warstwie, model FNO jest w stanie jednocześnie filtrować globalnie sygnały w przestrzeni spektralnej oraz modelować złożone, nieliniowe interakcje pomiędzy poszczególnymi składowymi częstotliwościowymi. Iteracyjne stosowanie powyższej struktury umożliwia aproksymację skrajnie nieliniowych operatorów, co stanowi kluczową przewagę nad klasycznymi metodami numerycznymi.

Transformacja wejścia (Lifting)

Przed wprowadzeniem danych w cykl warstw spektralnych, sygnał wejściowy $u(x)$ poddawany jest procesowi tzw. *podnoszenia wymiarowości* (ang. *lifting*). Wejście, które często posiada niską wymiarowość (np. pojedynczy parametr warunku początkowego), jest mapowane do przestrzeni o znacznie wyższej liczbie kanałów (ukrytej wymiarowości d_v) za pomocą płytkiej sieci neuronowej lub transformacji liniowej P:

$$v_0(x) = P(u(x)). \tag{3.8}$$

Ten krok pozwala modelowi na zakodowanie lokalnych cech sygnału w bogatszej reprezentacji wektorowej. Dopiero tak przetworzony, wysokowymiarowy stan ukryty v_0 jest przekazywany do kolejnych warstw spektralnych, gdzie podlega globalnej transformacji. Proces ten można interpretować jako rzutowanie surowych danych wejściowych do „przestrzeni cech", w której nieliniowe zależności między funkcjami stają się znacznie łatwiejsze do wyodrębnienia przez kolejne warstwy filtrujące.

Rozdział 4

Eksperymenty

W niniejszym rozdziale przedstawiono część eksperymentalną pracy oraz analizę wyników uzyskanych dla wybranych architektur sieci neuronowych. Omówiono rozważane problemy różniczkowe, sposób przygotowania danych treningowych, a następnie porównano jakość aproksymacji modeli na podstawie metryk błędu oraz wizualizacji predykcji.

4.1. Analizowane problemy analityczne

Do badania efektywności operatorów neuronowych wykorzystano zestaw równań różniczkowych: dwa zwyczajne oraz jedno cząstkowe. Rozważane równania reprezentują różne klasy problemów dynamicznych występujących w naukach ekonomicznych, matematyce stosowanej i fizyce matematycznej. Obejmują one zarówno modele opisujące procesy wzrostu w czasie, jak i równania opisujące nieliniową dynamikę przestrzenno-czasową. Taki dobór pozwala na analizę możliwości operatorów neuronowych w zadaniach o różnym stopniu złożoności oraz różnej strukturze danych wejściowych i wyjściowych. Dane dla każdego z równań wygenerowano numerycznie z wykorzystaniem bibliotek `numpy` oraz `scipy`.

4.1.1. Model wzrostu logistycznego

Model wzrostu logistycznego opisuje wzrost populacji w warunkach ograniczonych zasobów. Równanie przyjmuje postać [18, s. 2–5]:

$$\frac{dL(t)}{dt} = aL(t)\left(1 - \frac{L(t)}{K}\right) \tag{4.1}$$

i można je przekształcić do postaci jawnie nieliniowej:

$$\frac{dL(t)}{dt} = aL(t) - bL(t)^2, \tag{4.2}$$

gdzie:

- a – współczynnik wzrostu, określający tempo wzrostu populacji przy niskiej liczebności,
- b – współczynnik hamowania, reprezentujący wpływ zagęszczenia populacji na ograniczenie tempa wzrostu,
- $K = a/b$ – pojemność środowiska, czyli maksymalna wielkość populacji, jaką może utrzymać dane środowisko.

Model zawiera dwa punkty równowagi: $L(t) = 0$ (stan nietrwały) oraz $L(t) = K$ (punkt stabilny). Operator neuronowy uczy się mapowania parametrów wejściowych $X = [L(0), a, b]$ na ciągłą funkcję $L(t)$ w przedziale $T = 100$. Dane treningowe wygenerowano metodą Rungego-Kutty dla losowych parametrów: $a \in [0.1, 0.6]$, $b \in [0.001, 0.01]$ oraz $L(0) \in [5.0, 50.0]$. Mimo istnienia rozwiązania analitycznego, model ten służy jako fundament walidacji dla prostego nieliniowego operatora.

4.1.2. Model wzrostu RCK

Model Ramsey-Cass-Koopmans (RCK) [19, s. 58–65] opisuje ścieżki konsumpcji $c(t)$ i akumulacji kapitału $k(t)$ w czasie:

$$\dot{k}(t) = f(k(t)) - (n + g + \delta)k(t) - c(t), \tag{4.3}$$

$$\frac{\dot{c}(t)}{c(t)} = \frac{1}{\theta}\left[f'(k(t)) - \delta - \rho - \theta g\right], \tag{4.4}$$

gdzie $k(t)$ oznacza kapitał na jednostkę efektywnej pracy, $c(t)$ konsumpcję na jednostkę efektywnej pracy, a $f(k(t))$ jest funkcją produkcji (Cobba-Douglasa). Parametry $n, g, \delta, \rho, \theta$ określają odpowiednio stopy: wzrostu populacji, postępu technicznego, deprecjacji kapitału, preferencji czasowych oraz elastyczności substytucji międzyokresowej.

Pierwsze równanie odnosi wzrost kapitału do produkcji pomniejszonej o konsumpcję i inwestycje odtworzeniowe, drugie zaś opisuje dynamikę konsumpcji w odniesieniu do krańcowego produktu kapitału. Warunki początkowe losowano z przedziału $\pm 20\%$ otoczenia stanu stacjonarnego. Wyjściem operatora jest para trajektorii $(c(t), k(t))$, wymagająca wymiaru warstwy wyjściowej $2 \times N_t$, gdzie $N_t = 100$.

4.1.3. Równanie Burgersa

Równanie Burgersa jest nieliniowym równaniem różniczkowym cząstkowym, stanowiącym model dynamiki płynów:

$$\frac{\partial u(x,t)}{\partial t} + u(x,t)\frac{\partial u(x,t)}{\partial x} = \nu \frac{\partial^2 u(x,t)}{\partial x^2}, \tag{4.5}$$

gdzie x, t to odpowiednio położenie i czas, $u(x,t)$ – prędkość płynu – jest funkcją rozwiązania, ν współczynnikiem lepkości, natomiast człon $u\frac{\partial u}{\partial x}$ odpowiada za konwekcję, a $\nu\frac{\partial^2 u}{\partial x^2}$ za dyfuzję.

Operator neuronowy uczy się mapowania $[u(x,0), x, t] \to u(x,t)$. Warunek początkowy definiowano jako $u(x,0) = A\sin(k\pi x)$, gdzie parametry losowano z przedziałów: $A \in [0.5, 1.2]$, $k \in \{1, 2\}$, $\nu \in [0.02, 0.05]$. Dane wygenerowano za pomocą schematu różnic skończonych. Ze względu na konkurencję między konwekcją a dyfuzją, rozwiązanie może tworzyć bardzo ostre gradienty, co czyni ten problem wymagającym testem stabilności i zdolności generalizacyjnych operatorów neuronowych.

4.2. Przedstawienie otrzymanych wyników

Aby porównać działanie architektur w praktyce, dla każdego z równań zbudowano następujące modele: klasyczną sieć MLP, operator neuronowy Fouriera (FNO), standardowy DeepONet oraz DeepONet z warstwami konwolucyjnymi (CNN) w sieci *Branch*. Modele zaimplementowano w środowisku `PyTorch`, wykorzystując bibliotekę `neuraloperator` [20]. Hiperparametry dobrano za pomocą techniki *random search*, optymalizując m.in. liczbę kanałów ukrytych, liczbę modów Fouriera, współczynnik uczenia oraz parametry regularyzacji L^2. Dane wejściowe poddano standaryzacji `StandardScaler` zgodnie ze wzorem $Z = \frac{x-\mu}{\sigma}$.

Dla modeli wzrostu (logistyczny, RCK) wykorzystano błąd średniokwadratowy (MSE). W przypadku równania Burgersa, ze względu na silną nieliniowość i obecność ostrych gradientów, zastosowano względny błąd L^2:

$$E_{L^2} = \frac{\sqrt{\sum_{i=1}^{N}(u_i - \hat{u}_i)^2}}{\sqrt{\sum_{i=1}^{N} u_i^2}}, \tag{4.6}$$

gdzie u_i oznacza rozwiązanie referencyjne, a $\hat{u}_i$ predykcję modelu. Zastosowanie tej metryki uniezależnia ocenę dokładności od skali wartości lokalnych.

4.2.1. Model wzrostu logistycznego

Przed przystąpieniem do procesu trenowania dla każdej architektury przeprowadzono dobór hiperparametrów metodą *random search*. Analizowane hiperparametry oraz zakresy ich wartości przedstawiono w Tabeli 4.1, a najlepsze znalezione konfiguracje w Tabeli 4.2.

Model	**Hiperparametr i rozważane wartości**
MLP	`hidden_dim` $\in \{32, 64, 128, 256\}$, `lr` $\in \{10^{-3}, 5 \cdot 10^{-4}, 10^{-4}, 5 \cdot 10^{-5}\}$, `weight_decay` $\in \{0, 10^{-5}, 10^{-4}\}$, `optimizer` $\in$ {Adam,AdamW}
FNO	`hidden_channels` $\in \{64, 128, 256\}$, `n_modes` $\in \{16, 32, 48\}$, `lr` $\in \{10^{-4}, 5 \cdot 10^{-4}, 10^{-3}, 5 \cdot 10^{-3}\}$, `batch_size` $\in \{16, 32, 64\}$
DeepONet	`p` $\in \{64, 128, 256, 512\}$, `hidden_dim` $\in \{32, 64, 128, 256\}$, `lr` $\in \{10^{-3}, 5 \cdot 10^{-4}, 10^{-4}, 5 \cdot 10^{-5}\}$, `weight_decay` $\in \{0, 10^{-5}, 10^{-4}\}$, `optimizer` $\in$ {Adam, AdamW}
DeepONet + conv.	`p` $\in \{64, 128, 256, 512\}$, `hidden_dim` $\in \{32, 64, 128, 256\}$, `lr` $\in \{10^{-3}, 5 \cdot 10^{-4}, 10^{-4}, 5 \cdot 10^{-5}\}$, `weight_decay` $\in \{0, 10^{-5}, 10^{-4}\}$, `optimizer` $\in$ {Adam, AdamW}

Tabela 4.1: Przestrzeń hiperparametrów analizowana dla modelu wzrostu logistycznego.

W analizowanych modelach rozmiary warstw ukrytych: `p`, `hidden_dim`, `hidden_channels` odpowiadają za pojemność reprezentacyjną sieci. Zwiększenie ich wartości pozwala modelowi odwzorowywać bardziej złożone zależności, lecz jednocześnie zwiększa liczbę parametrów i ryzyko przeuczenia. Współczynnik uczenia `lr` kontroluje szybkość aktualizacji wag, natomiast parametr `weight_decay` odpowiada za regularyzację L^2. W przypadku FNO parametr `n_modes` określa liczbę zachowywanych modów Fouriera, wpływając bezpośrednio na zdolność modelu do reprezentowania struktur o różnych częstotliwościach.

Model	**Najlepsza konfiguracja**
MLP	`hidden_dim` $= 64$, `lr` $= 5 \cdot 10^{-4}$, `weight_decay` $= 10^{-4}$, `optimizer`=AdamW
FNO	`hidden_channels` $= 128$, `n_modes` $= 32$, `lr` $= 5 \cdot 10^{-3}$, `batch_size` $= 16$
DeepONet	`p` $= 256$, `hidden_dim` $= 64$, `lr` $= 5 \cdot 10^{-4}$, `weight_decay` $= 10^{-4}$, `optimizer`=AdamW
DeepONet + conv.	`p` $= 256$, `hidden_dim` $= 256$, lr$= 10^{-3}$, `weight_decay` $= 10^{-5}$, `optimizer`=Adam

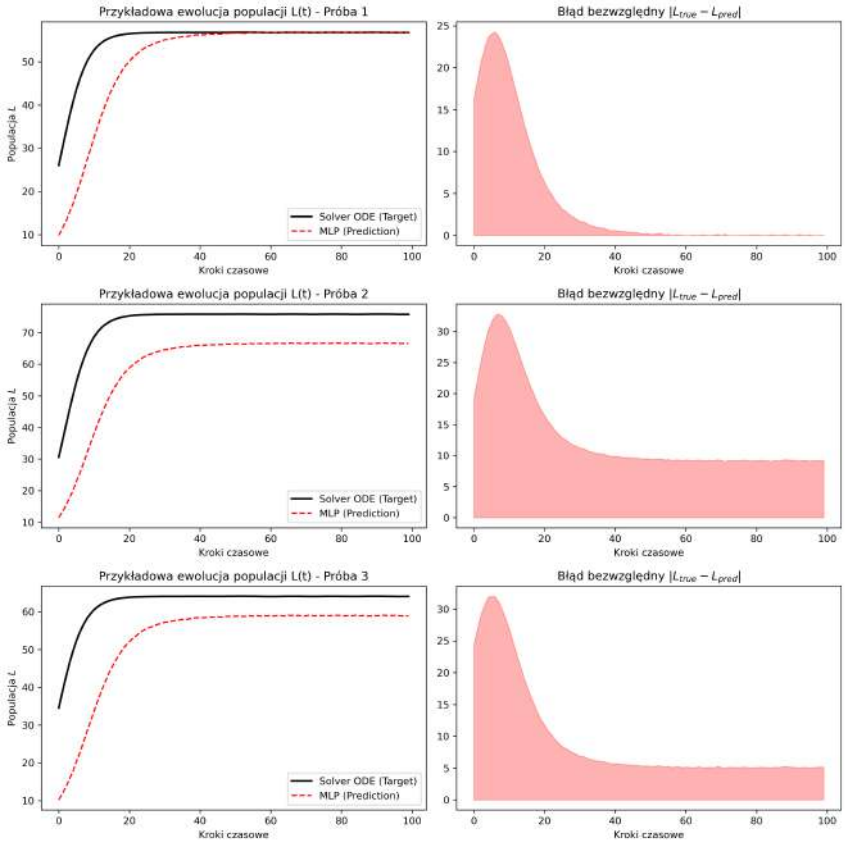
Tabela 4.2: Najlepsze konfiguracje hiperparametrów dla modelu wzrostu logistycznego.

Wyniki MSE przedstawione w Tabeli 4.3 jednoznacznie wskazują na wyższość architektur operatorowych nad klasycznym perceptronem.

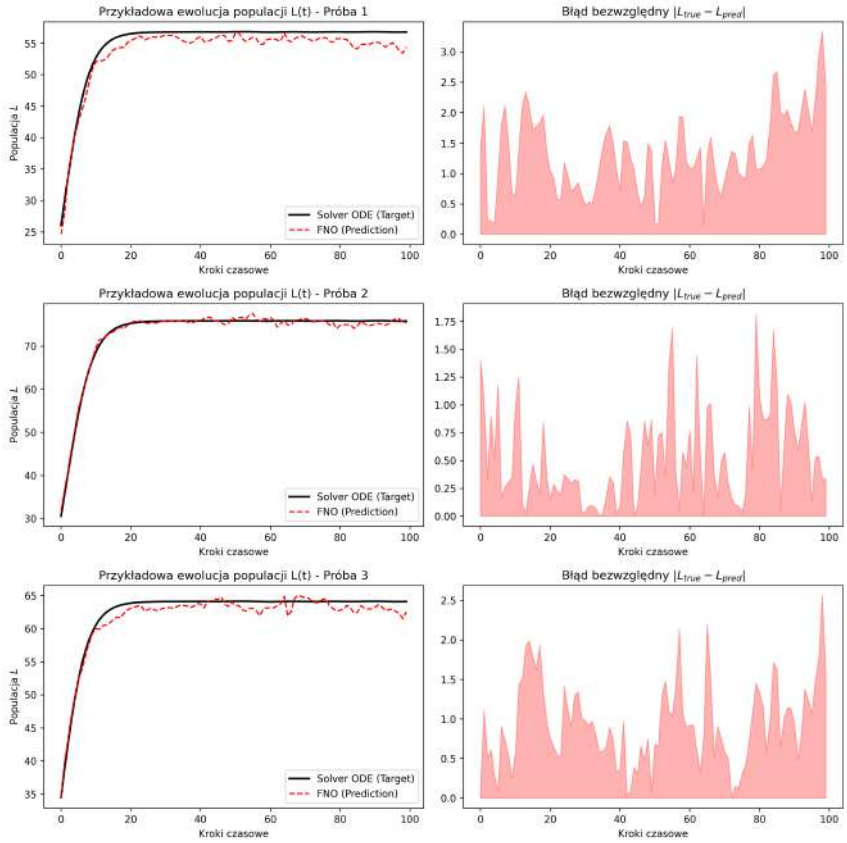
Architektura	**MLP**	**FNO**	**DeepONet**	**DeepONet + conv.**
Błąd MSE	732,89	24,30	77,66	51,41

Tabela 4.3: Błąd średniokwadratowy (MSE) na zbiorze testowym dla modelu wzrostu logistycznego.

Klasyczna sieć MLP, mimo optymalizacji, nie jest w stanie wyuczyć się operatora rozwiązania, wykazując tendencję do „uśredniania" predykcji. Jak pokazano na Rysunku 4.1, model ignoruje indywidualne warunki początkowe, generując jedną, uśrednioną trajektorię, co skutkuje wysokim błędem bezwzględnym przekraczającym 30 jednostek w początkowej fazie ewolucji populacji.

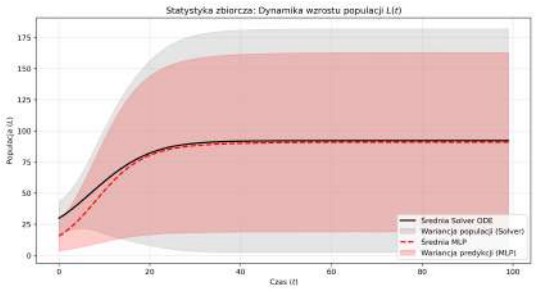


Rysunek 4.1: Niezdolność sieci MLP do generalizacji: model generuje niemal identyczną trajektorię dla różnych danych wejściowych.

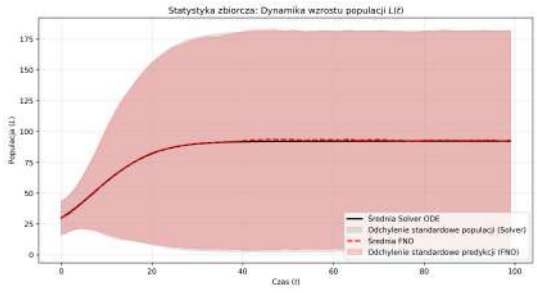


Rysunek 4.2: Wizualizacja predykcji sieci FNO: wysoka adaptacyjność do zmiennych warunków początkowych.

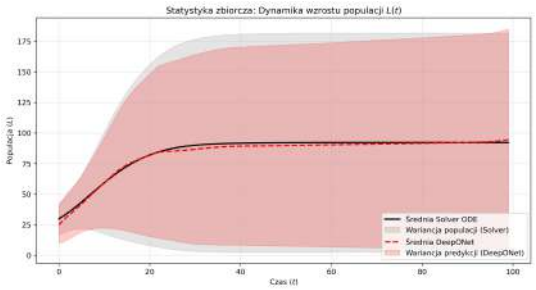
W przeciwieństwie do MLP, model FNO (Rysunek 4.2) adaptuje się do parametrów wejściowych, wiernie odwzorowując krzywe wzrostu z odchyleniami rzędu 1,5–3 jednostek. Analiza statystyczna (Rysunek 4.3) potwierdza wnioski ilościowe. Model FNO prezentuje najwyższą zdolność generalizacyjną, wiernie oddając zarówno średnią predykcję, jak i pasma odchyleń standardowych. Klasyczny DeepONet osiąga wyniki zbliżone do poprawne, jednak z niewielkimi niedokładnościami w fazie inicjalnej. Wprowadzenie warstw konwolucyjnych do DeepONet paradoksalnie pogorszyło stabilność długoterminową, prowadząc do zaniżania wariancji w późniejszych fazach ewolucji systemu.



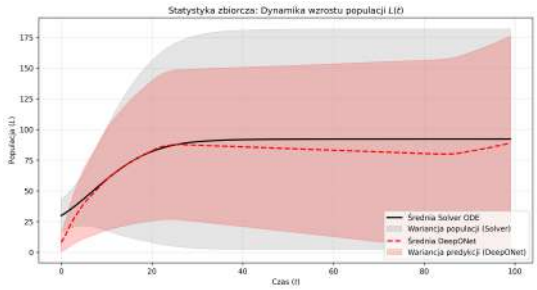
(a) MLP



(b) FNO



(c) DeepONet



(d) DeepONet + conv.

Rysunek 4.3: Zbiorcze zestawienie statystyk dynamiki wzrostu populacji dla wszystkich analizowanych architektur.

4.2.2. Model wzrostu RCK

W przypadku problemu RCK analizowano przestrzeń hiperparametrów przedstawioną w Tabeli 4.5. Ze względu na większą złożoność problemu, obejmującego jednoczesne modelowanie trajektorii kapitału i konsumpcji, dla poszczególnych architektur rozważano odmienne zestawy parametrów związanych z ich budową. Najlepsze konfiguracje dla poszczególnych architektur przedstawione są w Tabeli 4.6.

Tak jak w poprzednim równaniu, pod względem wartości błędu średniokwadratowego (Tabela 4.4) najlepiej poradziła sobie sieć FNO, architektury DeepOnet miały podobne wyniki, wersja z warstwą konwolucji była trochę gorsza, a na ostatnim miejscu o kilka rzędów dziesiętnych wypadła sieć MLP.

Architektura	**MLP**	**FNO**	**DeepOnet**	**DeepOnet + conv.**
Błąd MSE	0,021	0,0004	0,0026	0,0073

Tabela 4.4: Błąd średniokwadratowy (MSE) na danych testowych dla analizowanych architektur sieci neuronowych – model RCK.

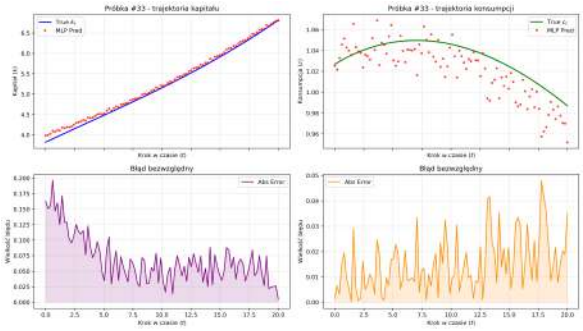
Model	**Hiperparametr i rozważane wartości**
MLP	`hidden_dims` $\in \{(256,256),(256,256,128),(512,256)\}$, `lr` $\in \{10^{-3}, 3\cdot10^{-4}\}$, `weight_decay` $\in \{0, 10^{-5}\}$, `batch_size` $\in \{64,128\}$
FNO	`n_modes` $\in \{8,12,16,24\}$, `hidden_channels` $\in \{32,48,64\}$, `projection_ratio` $\in \{1,2\}$, `lr` $\in \{5\cdot10^{-4}, 10^{-3}, 3\cdot10^{-3}\}$, `weight_decay` $\in \{10^{-5}, 10^{-4}\}$
DeepONet	`p` $\in \{32,64,128\}$, `branch_hidden_dim` $\in \{64,128,256\}$, `trunk_hidden_dim` $\in \{64,128,256\}$, `lr` $\in \{10^{-3}, 3\cdot10^{-4}, 10^{-4}\}$, `weight_decay` $\in \{10^{-5}, 10^{-4}\}$
DeepONet + conv.	`p` $\in \{16,32,64\}$, `branch_channels` $\in \{8,16,24\}$, `trunk_hidden_dim` $\in \{32,64,128\}$, `lr` $\in \{10^{-3}, 3\cdot10^{-3}\}$, `weight_decay` $\in \{10^{-5}, 10^{-4}\}$

Tabela 4.5: Przestrzeń hiperparametrów analizowana dla modelu RCK.

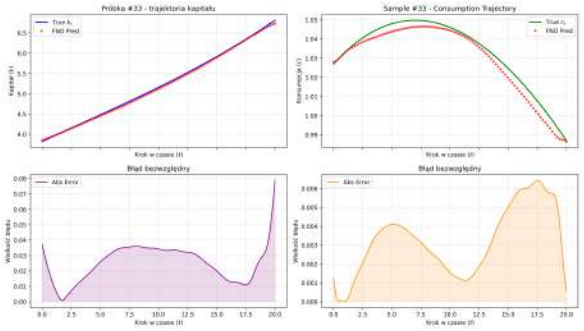
Model	**Najlepsza konfiguracja**
MLP	`hidden_layers` $= (256,256)$, `lr` $= 10^{-3}$, `weight_decay` $= 0$, `batch_size` $= 64$
FNO	`n_modes` $= 24$, `hidden_channels` $= 64$, `projection_ratio` $= 2$, `lr` $= 3\cdot10^{-3}$, `weight_decay` $= 10^{-5}$
DeepONet	`p` $= 64$, `branch_hidden_dim` $= 256$, `trunk_hidden_dim` $= 128$, `lr` $= 10^{-3}$, `weight_decay` $= 10^{-4}$
DeepONet + conv.	`p` $= 32$, `branch_channels` $= 16$, `trunk_hidden_dim` $= 32$, `lr` $= 3\cdot10^{-3}$, `weight_decay` $= 10^{-5}$

Tabela 4.6: Najlepsze konfiguracje hiperparametrów dla modelu RCK.

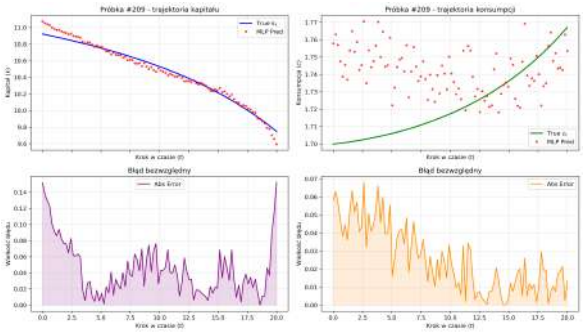
Przechodząc do analizy poszczególnych próbek (Rysunek 4.4 i 4.5), wnioski są zbliżone do tych z modelu wzrostu logistycznego. W przypadku sieci MLP ponownie uwidacznia się jej tendencja do uśredniania trajektorii. Choć dla próbki 33 (4.4a) przebieg predykcji jest zbliżony do referencyjnego, to analiza próbek 209 (4.4c) i 280 (4.4e) pokazuje brak elastyczności modelu – sieć generuje zbliżone odpowiedzi niezależnie od warunków początkowych. Przekłada się to bezpośrednio na wysokie wartości błędu bezwzględnego.



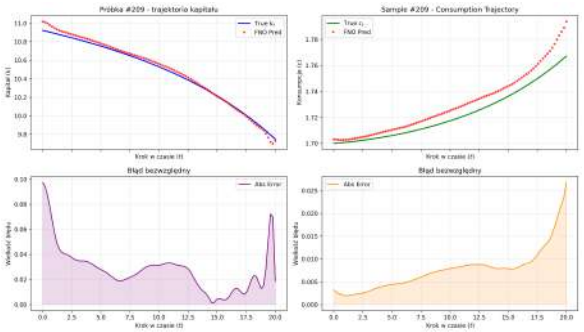
(a) Próbka 33 – MLP



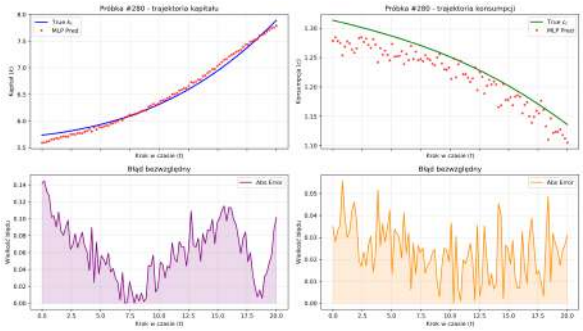
(b) Próbka 33 – FNO



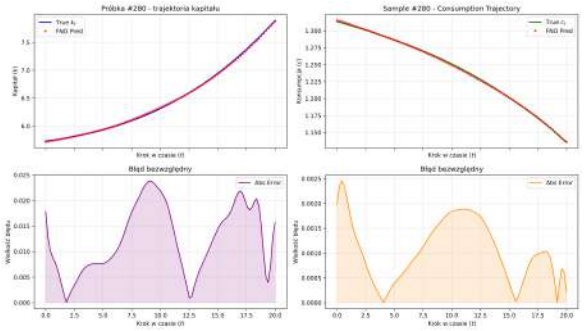
(c) Próbka 209 – MLP



(d) Próbka 209 – FNO



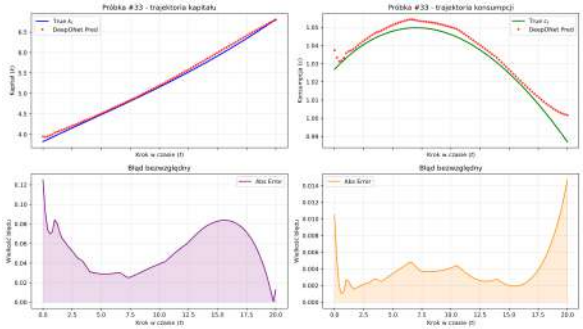
(e) Próbka 280 – MLP



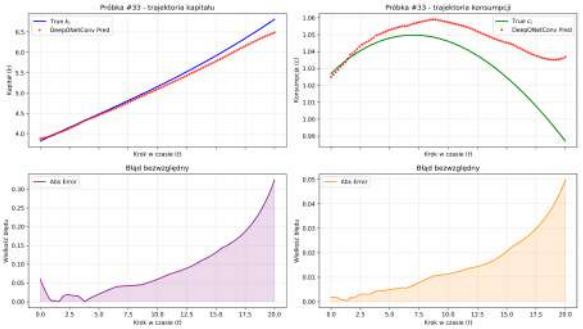
(f) Próbka 280 – FNO

Rysunek 4.4: Zestawienie predykcji trajektorii oraz błędów bezwzględnych w modelu RCK. Wiersze odpowiadają analizowanym próbkom testowym (33, 209, 280), natomiast kolumny przedstawiają wyniki dla poszczególnych architektur, kolejno: MLP i FNO.

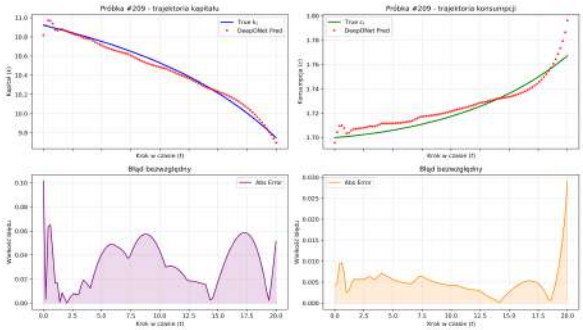
Natomiast, FNO wykazuje się największą dokładnością (4.4b, 4.4d, 4.4f), bezbłędnie dopasowując linię predykcji do prawdziwej trajektorii we wszystkich analizowanych przypadkach. Analiza wykresów błędu potwierdza, że odchylenia od trajektorii numerycznej są minimalne, stabilne w czasie i rzadko przekraczają wartości rzędu $0{,}2 - 0{,}4$ jednostki, co udowadnia doskonałe opanowanie mapowania nieskończenie wymiarowego w dziedzinie częstotliwościowej.



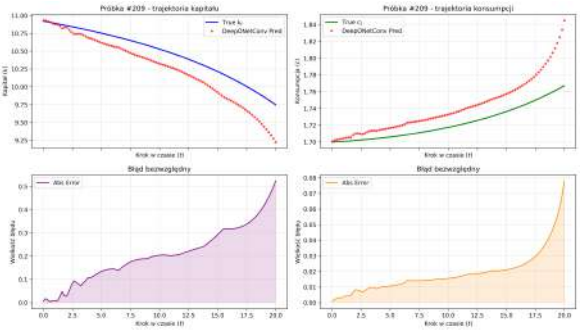
(a) Próbka 33 – DeepONet



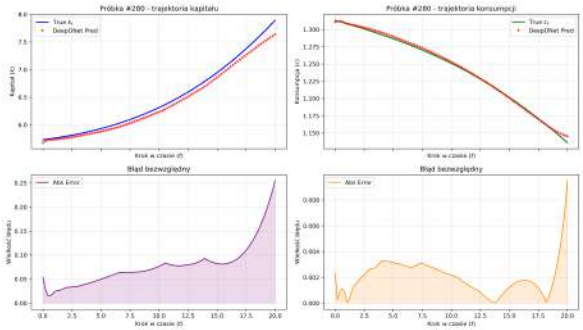
(b) Próbka 33 – DON + conv.



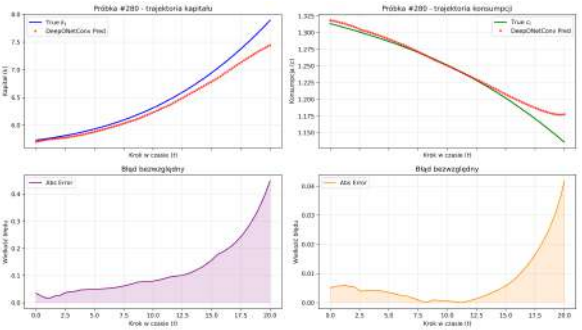
(c) Próbka 209 – DeepONet



(d) Próbka 209 – DON + conv.



(e) Próbka 280 – DeepONet



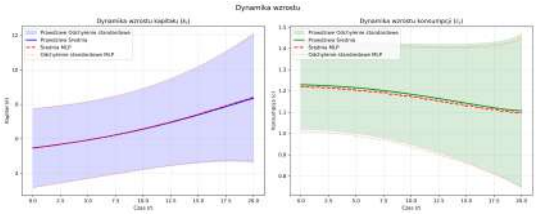
(f) Próbka 280 – DON + conv.

Rysunek 4.5: Zestawienie predykcji trajektorii oraz błędów bezwzględnych w modelu RCK. Wiersze odpowiadają analizowanym próbkom testowym (33, 209, 280), natomiast kolumny przedstawiają wyniki dla poszczególnych architektur, kolejno: DeepONet i DeepONet z konwolucją.

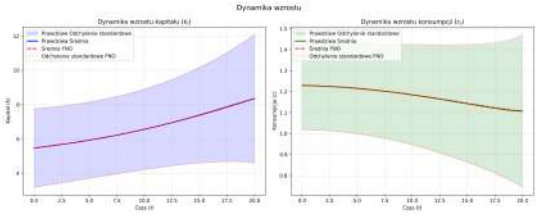
Architektura DeepONet w wariancie klasycznym (4.5a, 4.5c, 4.5e) również reprezentuje wysoki poziom dopasowania – trajektorie są odtwarzane płynnie. Jednak w porównaniu do FNO można zaobserwować odchylenia w punktach zwrotnych nieliniowej dynamiki oraz minimalnie wyższą amplitudę błędu bezwzględnego, sięgającą okolic $0{,}5 - 1{,}0$ jednostki, zwłaszcza w końcowych fazach symulacji.

Najbardziej interesujące zjawisko zachodzi znowu po wprowadzeniu warstw konwolucyjnych do sieci Branch (4.5b, 4.5d, 4.5f). W tym scenariuszu model wykazuje wyraźną niestabilność. Tak jak w modelu logistycznym, początkowe fazy predykcji pokrywają się z rozwią-

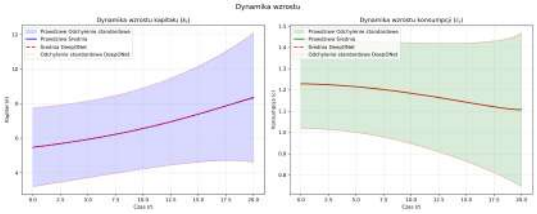
zaniem, to w miarę upływu czasu (kroki powyżej 40) sieć zaczyna odbiegać od prawdziwej trajektorii.



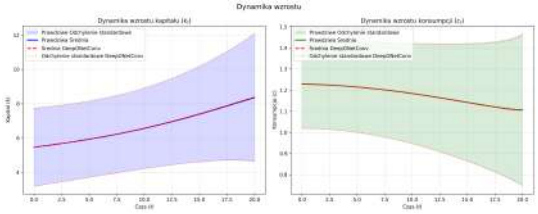
(a) Klasyczna sieć MLP



(b) Operator Neuronowy Fouriera (FNO)



(c) Klasyczny DeepONet



(d) DeepONet z warstwami konwolucyjnymi

Rysunek 4.6: Statystyka zbiorcza dynamiki wzrostu kapitału k_t oraz konsumpcji c_t w modelu RCK. Wykresy przedstawiają porównanie średniej predykcji oraz odchylenia standardowego z prawdziwymi wartościami.

Przy analizie, jak modele radziły sobie z rekonstrukcją rozkładu prawdopodobieństwa rozwiązań układu RCK (Rysunek 4.6), wnioski są bardzo podobne. Najsłabsze wyniki, szczególnie w kontekście dynamiki konsumpcji c_t, uzyskano dla sieci MLP. Choć średnia predykcji kapitału k_t jest relatywnie poprawna, to w przypadku konsumpcji czerwona linia przerywana (predykcja) wyraźnie odbiega od zielonej linii ciągłej (referencja). Co więcej, pasmo odchylenia standardowego wygenerowane przez MLP nie pokrywa się w wielu miejscach z obszarem rzeczywistej zmienności danych. Chociaż w agregacji całego zbioru sieć MLP wypada lepiej niż przy analizie poszczególnych próbek, zdecydowanie bardziej widać problemowość architektury na pojedynczych przykładach.

Wszystkie architektury operatorowe bardzo dobrze nauczyły się zarówno średniej trajektorii, jak i rozpiętości wyników w czasie. Biorąc pod uwagę błąd MSE, pojedyncze trajektorie oraz statystki zbiorcze, to jasno widać jak dużo lepiej nauczyły się rozwiązania architektury operatorowe, a w szczególności model FNO, który we wszystkich porównaniach wypadł najlepiej.

4.2.3. Równanie Burgersa

Równanie Burgersa jest bardziej wymagającym problemem niż wcześniej analizowane równania zwyczajne, ponieważ rozwiązanie zależy jednocześnie od czasu i przestrzeni. Modele muszą więc nauczyć się odwzorowania pomiędzy funkcjami zdefiniowanymi na siatce dwuwymiarowej (x, t), a dodatkową trudność stanowi obecność ostrych gradientów oraz propagujących się fal. Wszystkie architektury trenowano na tym samym zbiorze danych oraz tej samej siatce przestrzenno–czasowej. Jako główną metrykę jakości przyjęto względny błąd normy L^2 (ang. *relative* L^2 *error*), który lepiej niż błąd średniokwadratowy odzwierciedla jakość rekonstrukcji całego pola rozwiązania.

Analizowana przestrzeń hiperparametrów przedstawiona jest w Tabeli 4.7, a najlepsze konfiguracje dla poszczególnych architektur przedstawione są w Tabeli 4.8.

Model	**Hiperparametr i rozważane wartości**
MLP	`hidden_layers` $\in \{(16,16),(32,32)\}$, `lr` $\in \{10^{-3}, 5\cdot10^{-4}, 3\cdot10^{-4}\}$, `epochs`$= 5$, `weight_decay` $\in \{0, 10^{-5}, 10^{-4}\}$
FNO	`n_modes` $\in \{(8,8),(12,12),(16,16),(24,16)\}$, `hidden_channels` $\in \{16,32,64\}$, `n_layers` $\in \{2,3,4\}$, `lr` $\in \{10^{-3}, 3\cdot10^{-3}, 5\cdot10^{-3}\}$, `weight_decay` $\in \{0, 10^{-6}, 10^{-4}\}$
DeepONet	`branch_hidden_dim` $\in \{64,128,256\}$, `trunk_hidden_dim` $\in \{32,64,128\}$, `trunk_layers` $\in \{2,3,4\}$, `embedding_dim` $\in \{32,64,96\}$, `lr` $\in \{10^{-3}, 2\cdot10^{-3}, 5\cdot10^{-3}\}$, `weight_decay` $\in \{0, 10^{-6}, 10^{-4}\}$
DeepONet + conv.	`branch_filters` $\in \{16,32,48\}$, `trunk_hidden_dim` $\in \{32,64,128\}$, `trunk_layers` $\in \{2,3,4\}$, `embedding_dim` $\in \{32,64,96\}$, `lr` $\in \{10^{-3}, 2\cdot10^{-3}, 5\cdot10^{-3}\}$, `weight_decay` $\in \{0, 10^{-6}, 10^{-4}\}$

Tabela 4.7: Przestrzeń hiperparametrów analizowana dla równania Burgersa.

Model	**Najlepsza konfiguracja**
MLP	`hidden_layers` $= (32, 32)$, `lr` $= 10^{-3}$, `epochs` $= 5$, `weight_decay` $= 0$
FNO	`n_modes` $= (16, 16)$, `hidden_channels` $= 64$, `n_layers` $= 3$, `lr` $= 5 \cdot 10^{-3}$, `weight_decay` $= 10^{-6}$
DeepONet	`branch_hidden_dim` $= 256$, `trunk_hidden_dim`$= 128$, `trunk_layers` $= 2$, `embedding_dim` $= 96$, `lr` $= 5 \cdot 10^{-3}$, `weight_decay` $= 0$
DeepONet + conv.	`branch_filters` $= 16$, `trunk_hidden_dim` $= 128$, `trunk_layers` $= 2$, `embedding_dim` $= 96$, `lr` $= 5 \cdot 10^{-3}$, `weight_decay` $= 0$

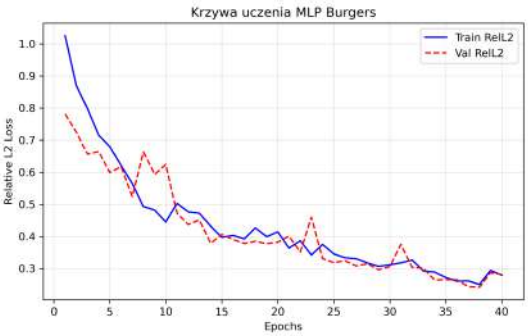
Tabela 4.8: Najlepsze konfiguracje hiperparametrów dla równania Burgersa.

Biorąc pod uwagę samą wartość metryki (Tabela 4.9), najlepiej ponownie sprawdziła się architektura FNO, jednak niewiele gorzej poradził sobie DeepONet z warstwami konwolucyjnymi. Na kolejnym miejscu znalazł się klasyczny DeepONet, natomiast zdecydowanie najwyższy błąd osiągnęła klasyczna sieć MLP.

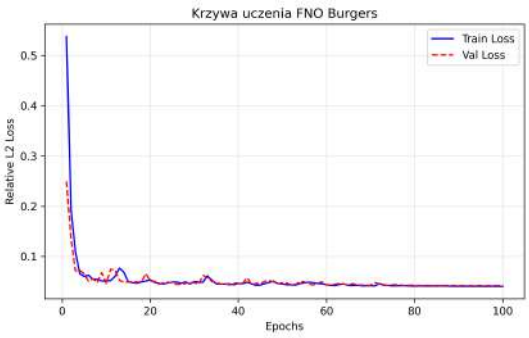
Architektura	**MLP**	**FNO**	**DeepONet**	**DeepONet + conv.**
Względne L^2	0,28107	0,04197	0,05937	0,04806

Tabela 4.9: Względny błąd normy L^2 na zbiorze testowym dla analizowanych architektur w zadaniu rozwiązania równania Burgersa.

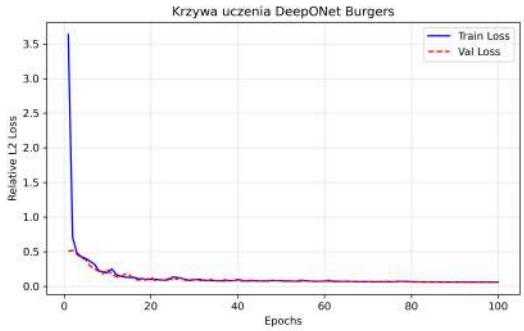
W przeciwieństwie do operatorów neuronowych, które przetwarzają całe trajektorie jednocześnie, klasyczna sieć MLP była uczona punktowo. Każda pojedyncza obserwacja miała postać $(u_0(x), x, t)$, gdzie $u_0(x)$ oznacza wartość warunku początkowego, natomiast celem sieci było przewidzenie wartości rozwiązania $u(x, t)$. Ponieważ zarówno zmienna przestrzenna x, jak i czasowa t były dyskretyzowane na siatce, całkowita liczba obserwacji rosła proporcjonalnie do liczby punktów siatki przestrzenno–czasowej. W rezultacie zbiór treningowy zawierał miliony punktów uczących, co wymusiło zastosowanie dużych rozmiarów batchy, ostatecznie równych 16384. Wyniki te pokazują, że klasyczna architektura MLP nie jest szczególnie efektywnym podejściem do aproksymacji operatorów dla problemów PDE, zwłaszcza w porównaniu z architekturami zaprojektowanymi bezpośrednio do pracy na przestrzeniach funkcji.



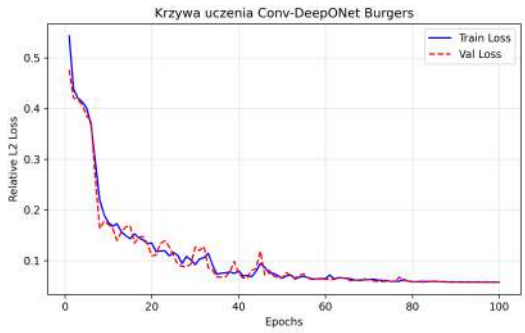
(a) Klasyczna sieć MLP



(b) Operator Neuronowy Fouriera (FNO)



(c) Klasyczny DeepONet

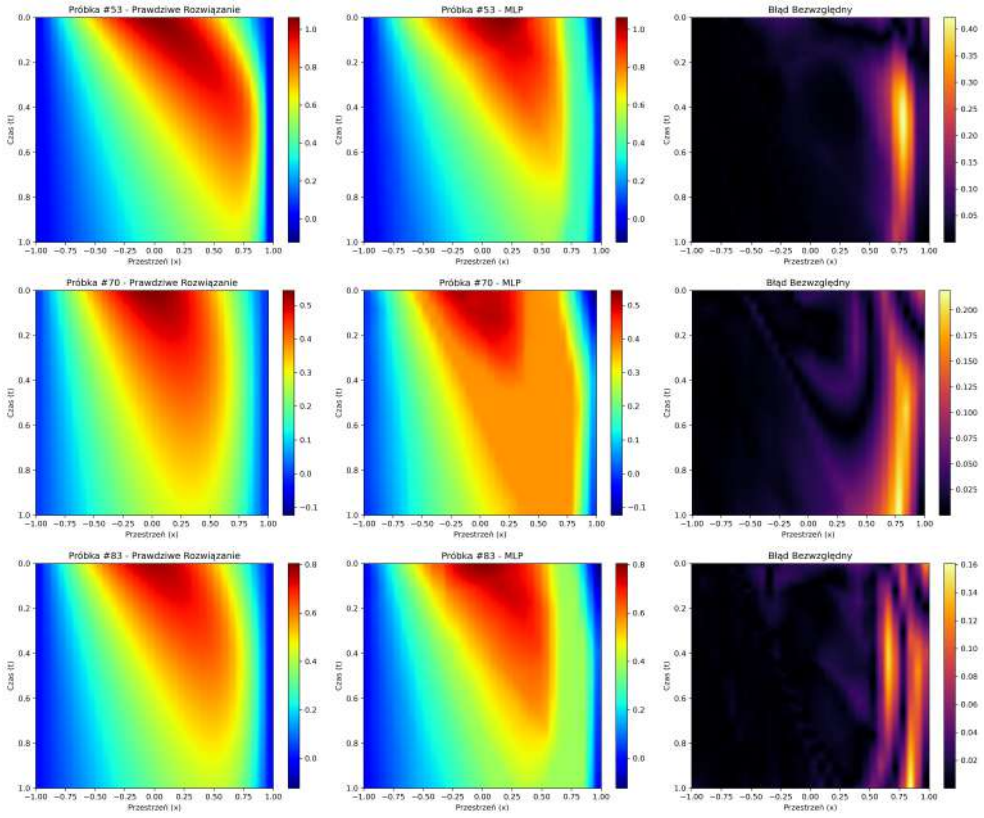


(d) DeepONet z warstwami konwolucyjnymi

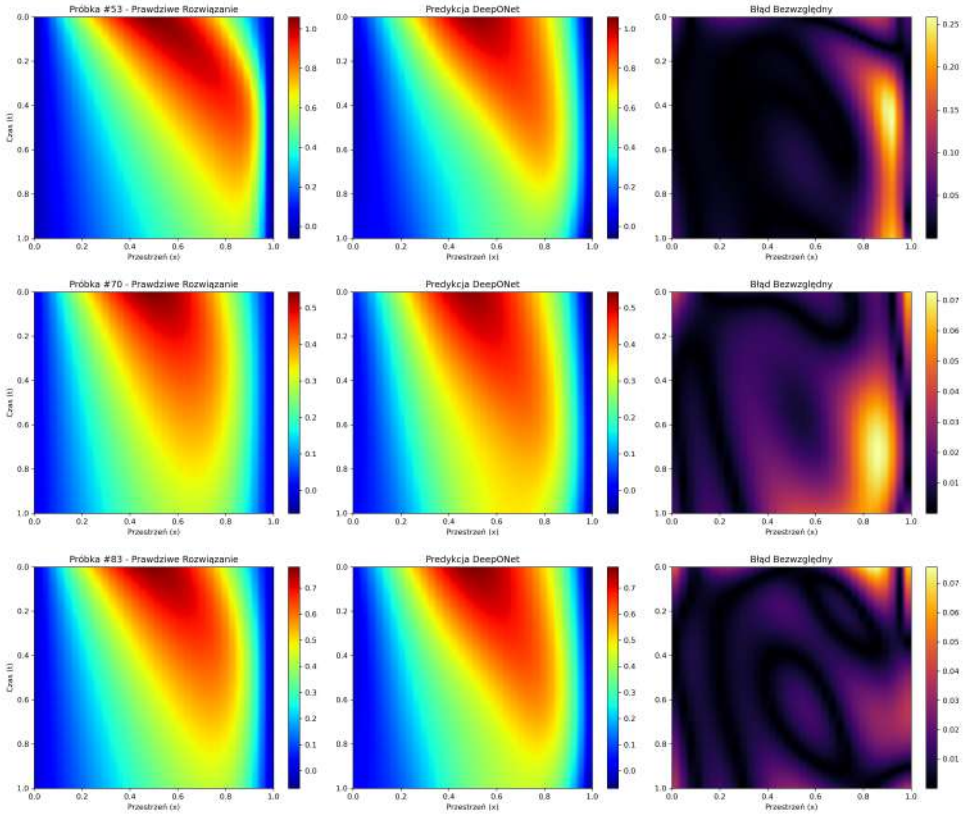
Rysunek 4.7: Przebieg procesu uczenia analizowanych modeli na zbiorze treningowym i walidacyjnym dla równania Burgersa.

Na podstawie krzywych uczenia dla wszystkich czterech architektur (Rysunek 4.7) można stwierdzić, że w żadnym przypadku nie wystąpiło przeuczenie modeli. Jednocześnie wyraźnie widać, że model MLP (4.7a) dopasowuje się do danych znacznie gorzej niż pozostałe architektury – spadek wartości funkcji straty jest zdecydowanie wolniejszy. W przypadku architektury FNO (4.7b) największą poprawę jakości predykcji można zaobserwować już w pierwszych pięciu epokach treningu.

Jak widać na Rysunku 4.8, klasyczna sieć MLP nauczyła się ogólnego kształtu rozwiązania, jednak model popełnia znaczące błędy, szczególnie w obszarach gwałtownych zmian wartości funkcji. Widoczne jest to również na mapach błędu bezwzględnego – największe błędy pojawiają się dokładnie w obszarach odpowiadających ostrym gradientom rozwiązania. W przypadku predykcji MLP można zauważyć, że rekonstruowane pole rozwiązania jest wyraźnie mniej gładkie, szczególnie dla próbki 70, co wskazuje na ograniczoną zdolność modelu do odwzorowania lokalnych struktur charakterystycznych dla dynamiki równania Burgersa. Warto zwrócić uwagę, że maksymalny błąd bezwzględny oscyluje w tym przypadku między 0.16 a 0.40.

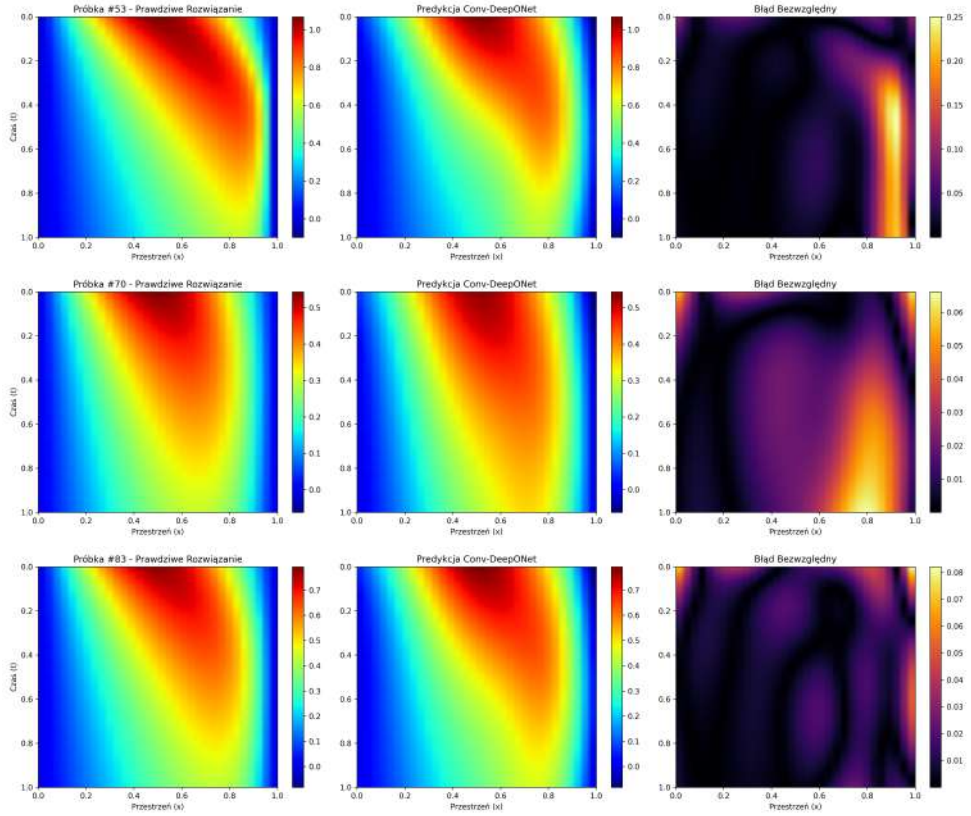


Rysunek 4.8: Porównanie rzeczywistego rozwiązania, predykcji oraz map błędu bezwzględnego dla architektury MLP.



Rysunek 4.9: Porównanie rzeczywistego rozwiązania, predykcji oraz map błędu bezwzględnego dla architektury DeepONet.

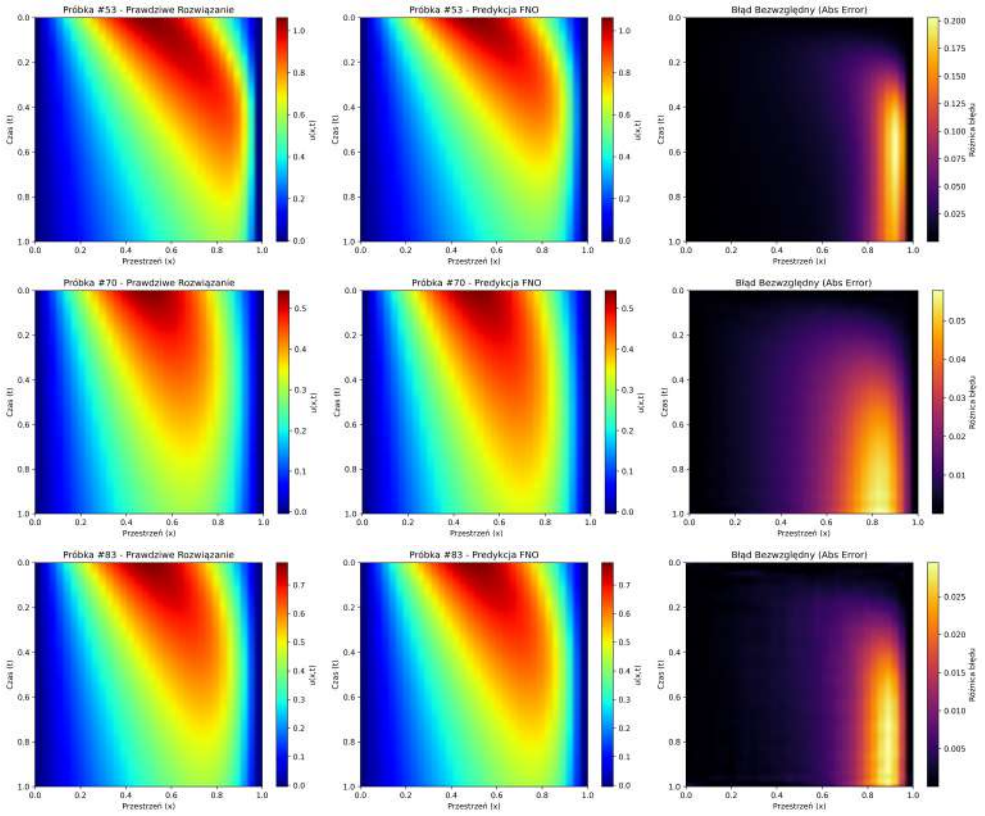
W przypadku architektury DeepONet jakość rekonstrukcji jest wyraźnie lepsza. Na Rysunku 4.9 można zauważyć znacznie niższe wartości błędu bezwzględnego – maksymalne wartości mieszczą się teraz w przedziale od około 0,06 do 0,25. Największe błędy nadal pojawiają się w obszarach dużych zmian wartości funkcji, jednak ich intensywność oraz rozmiar są wyraźnie mniejsze niż w przypadku klasycznej sieci MLP.



Rysunek 4.10: Porównanie rzeczywistego rozwiązania, predykcji oraz map błędu bezwzględnego dla DeepONet z warstwami konwolucyjnymi.

Architektura DeepONet z warstwami konwolucyjnymi osiągnęła wyniki bardzo zbliżone do klasycznego DeepONet. Maksymalne wartości błędu bezwzględnego pozostają w podobnych przedziałach, jednak lokalnie można zauważyć nieco lepsze odwzorowanie struktur przestrzennych oraz gładszy przebieg rekonstruowanego rozwiązania.

Z Rysunku 4.11 widać wyraźną poprawę jakości rekonstrukcji względem poprzednich modeli. Maksymalny błąd bezwzględny mieści się w przedziale od około 0,02 do 0,20, czyli na poziomie porównywalnym z architekturami DeepONet, choć średnio nadal nieco niższym. Największą różnicę można jednak zauważyć w strukturze map błędu bezwzględnego – jasne obszary nie są już rozproszone w całej domenie, lecz koncentrują się lokalnie w pobliżu obszaru $x \approx 0{,}8$ oraz dla wartości czasu $t < 0{,}3$. Oznacza to, że model FNO znacznie lepiej radzi sobie z globalną rekonstrukcją rozwiązania oraz propagacją struktur falowych w przestrzeni i czasie.



Rysunek 4.11: Porównanie rzeczywistego rozwiązania, predykcji oraz map błędu bezwzględnego dla architektury FNO.

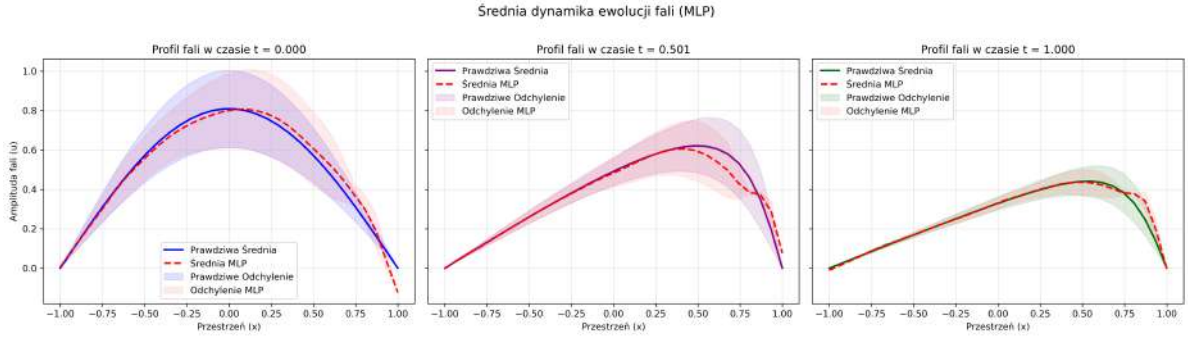
Na Rysunku 4.12 pokazane są różne przebiegi rekonstrukcji dynamiki rozwiązania równania Burgersa w czasie i przestrzeni. Każdy z trzech paneli (dla każdego z czterech modeli) przedstawia przekrój rozwiązania dla ustalonego czasu, odpowiednio od lewej do prawej: $t = 0$, $t = 0{,}5$ i $t = 1$. Oś x odpowiada przestrzeni, a oś y amplitudzie rozwiązania $u(x, t)$. Na każdym wykresie linia ciągła oznacza średnie rzeczywiste rozwiązanie, natomiast linia przerywana przedstawia średnią predykcję modelu. Kolorowe półprzezroczyste obszary pokazują odchylenia standardowe zarówno rzeczywistych danych, jak i predykcji modelu.

Zaczynając od klasycznej sieci MLP (4.12a), widać wyraźnie, że model najgorzej poradził sobie z rekonstrukcją dynamiki funkcji. Jest to szczególnie widoczne dla chwil $t = 0{,}5$ oraz $t = 1$: choć model częściowo uchwycił przesunięcie maksimum funkcji, to jednak wyraźnie zniekształca jej przebieg, co wynika z punktowego charakteru działania klasycznej sieci MLP oraz braku jawnego modelowania zależności operatorowych i przestrzennych. Natomiast dla warunku początkowego model ma tendencję do przewidywania wartości przesuniętych w przestrzeni x – predykcje są bardziej nachylone w prawą stronę niż rzeczywiste wartości, co widoczne jest zarówno dla średniej trajektorii, jak i odchylenia standardowego.

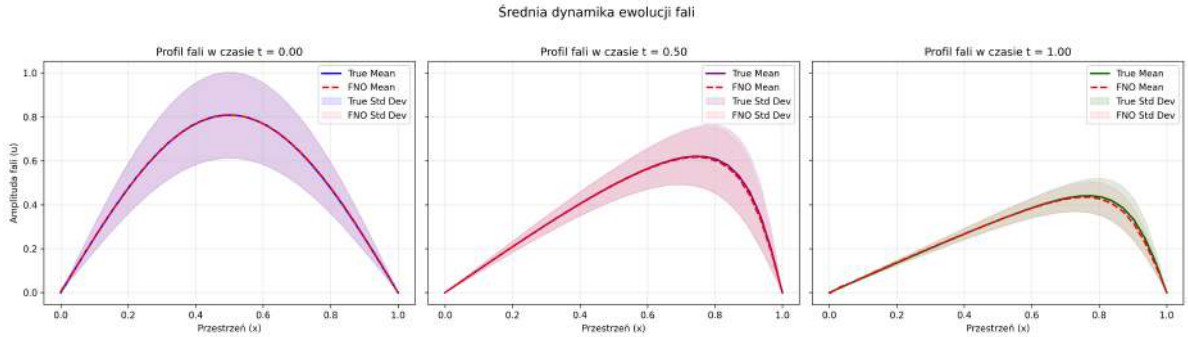
Architektura FNO (4.12b) ponownie poradziła sobie najlepiej z analizowanym problemem, ponieważ zarówno średnie predykcje, jak i odpowiadające im odchylenia standardowe niemal całkowicie pokrywają się z wartościami rzeczywistymi. Jedynie dla większych wartości czasu t można zauważyć niewielkie niedoszacowanie amplitudy rozwiązania u, jednak mimo tego kształt funkcji pozostaje bardzo dobrze zachowany, a różnice są praktycznie nieznaczące.

Na końcu, oba warianty architektury DeepONet (4.12c oraz 4.12d) osiągnęły bardzo zbliżone wyniki. Co najważniejsze, w przeciwieństwie do MLP, modele poprawnie zachowują kształt przewidywanej funkcji, a same predykcje pozostają bliskie wartościom rzeczywistym. Dla warunku początkowego $t = 0$ w obu modelach największe różnice pojawiają się przy brzegach domeny ($x \approx 0$ oraz $x \approx 1$), gdzie rozwiązanie jest lekko wygładzane. Odchylenia standardowe są również zbliżone do rzeczywistych, choć różnice są nieco większe dla klasycznego DeepONet.

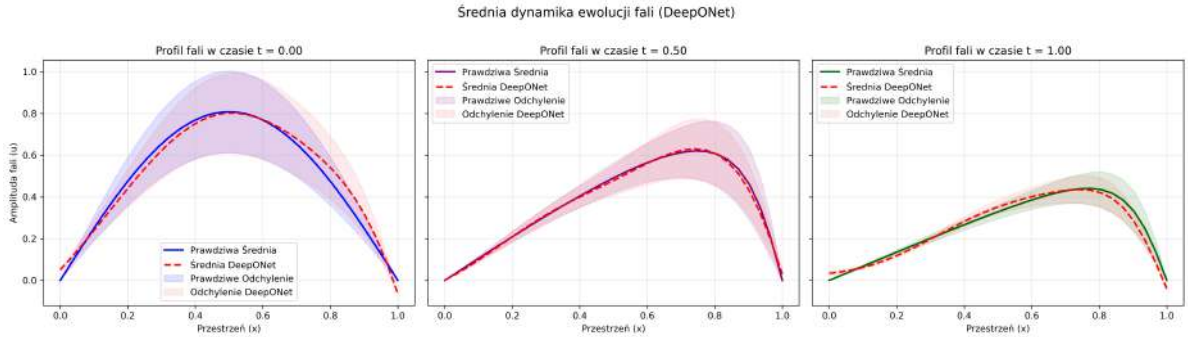
Dla chwili $t = 0{,}5$ obie architektury poprawnie uchwyciły przesunięcie maksimum funkcji oraz zachowały jej ogólny kształt. Największe błędy pojawiają się w obszarach stromych gradientów, szczególnie po prawej stronie maksimum amplitudy. Dla $t = 1$ widoczne jest zarówno dalsze przesunięcie maksimum, jak i obniżenie amplitudy rozwiązania, zgodnie z rzeczywistą dynamiką równania Burgersa. W obu przypadkach odchylenia standardowe predykcji nieco różnią się od rzeczywistych, co sugeruje, że modele dobrze nauczyły się średniego rozwiązania operatora, lecz gorzej odwzorowują pełną strukturę statystyczną trajektorii w obszarach gwałtownych zmian funkcji. Niezależnie od analizowanego czasu, można zauważyć, że obie architektury DeepONet mają największe trudności przy silnych zmianach gradientu rozwiązania, jednak mimo tego zachowują gładkość i poprawną strukturę przewidywanych profili.



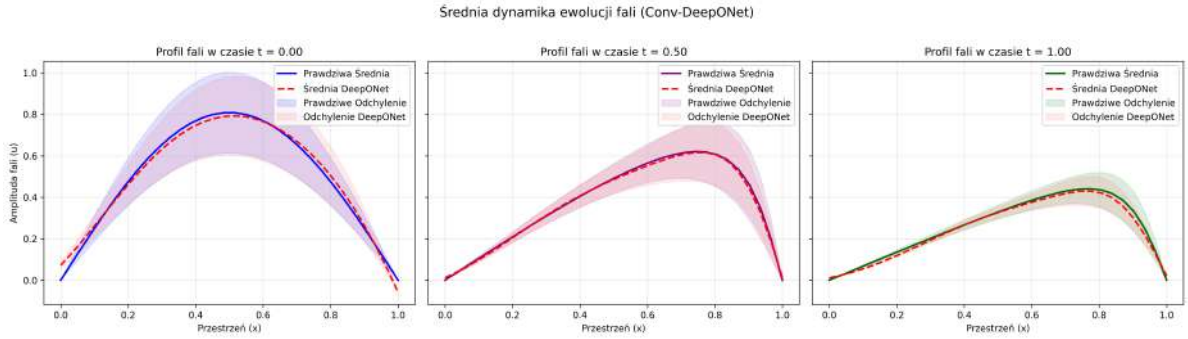
(a) Klasyczna sieć MLP.



(b) FNO.



(c) Klasyczny DeepONet.



(d) DeepONet z warstwami konwolucyjnymi

Rysunek 4.12: Średnia dynamika ewolucji fal w czasie.

Podsumowanie

Niniejsza praca magisterska została poświęcona szczegółowej analizie teoretycznej oraz ewaluacji eksperymentalnej nowoczesnych metod uczenia maszynowego ukierunkowanych na aproksymację operatorów matematycznych. Głównym celem badawczym było wykazanie przewagi architektur zaprojektowanych do pracy bezpośrednio na przestrzeniach nieskończenie wymiarowych – takich jak **Fourier Neural Operator (FNO)** oraz **DeepONet** – nad klasycznym podejściem opartym na wielowarstwowym perceptronie.

Synteza części teoretycznej

W warstwie teoretycznej praca ugruntowała matematyczne podstawy uczenia operatorów w oparciu o analizę funkcjonalną. Kluczowe znaczenie miało zdefiniowanie struktur przestrzeni Banacha i Hilberta oraz pojęcia regularności w sensie Sobolewa, co pozwoliło na formalny opis mapowań, w których zarówno argumentami, jak i wynikami są funkcje.

Następnie wskazano na ograniczenia systemów komputerowych, które nie operują bezpośrednio na obiektach ciągłych, co wymusza zastosowanie odpowiednich metod dyskretyzacji. Na tej bazie objaśniono dwie przełomowe architektury operatorowe:

- **DeepONet:** realizujący uniwersalną aproksymację poprzez system sieci gałęzi (*Branch Network*) kodującej sygnał wejściowy oraz sieci pnia (*Trunk Network*) budującej ciągłą, adaptacyjną bazę przestrzeni wyjściowej.
- **Fourier Neural Operator (FNO):** wykorzystujący twierdzenie o splocie i mapujący globalne zależności w przestrzeni spektralnej za pomocą szybkiej transformacji Fouriera (FFT).

Teoretyczne rozważania zamknęła klasyfikacja operatorów na liniowe i nieliniowe oraz lokalne (różniczkowe) i globalne (całkowe), co pozwoliło zrozumieć, dlaczego architektury spektralne i całkowe uczą się reprezentacji bezpośrednio z danych znacznie efektywniej niż metody numeryczne.

Wyniki eksperymentalne i ich interpretacja

Część empiryczna pracy obejmowała wytrenowanie czterech modeli (MLP, FNO, klasyczny DeepONet oraz DeepONet z warstwą konwolucyjną) dla trzech problemów różniczkowych o rosnącym stopniu złożoności:

1. **Model wzrostu logistycznego (ODE):** Wykazał podstawową słabość sieci MLP, która uśredniała trajektorie, ignorując specyficzne warunki początkowe. Modele operatorowe, szczególnie FNO, wiernie odwzorowywały dynamikę procesów.
2. **Model wzrostu Ramseya-Cass-Koopmansa (ODE):** Potwierdził wysoką skuteczność FNO. Wprowadzenie modyfikacji konwolucyjnej do DeepONet pogorszyło stabilność długoterminową modelu, co wskazuje, że w problemach typu ODE prostsze architektury DeepONet są bardziej odporne na błędy.
3. **Równanie Burgersa (PDE):** Najbardziej wymagający problem dwuwymiarowy. FNO osiągnęło najlepsze wyniki w odwzorowaniu ostrych gradientów. DeepONet z konwolucją wykazał zdolność do adaptacji do lokalnych struktur przestrzennych, natomiast MLP generowało skrajnie wysokie błędy w rejonach dużych zmian gradientu.

Przeprowadzone eksperymenty pozwalają pozytywnie zweryfikować postawioną hipotezę badawczą. We wszystkich analizowanych problemach operatory neuronowe osiągały wyższą dokładność predykcji niż klasyczna sieć MLP, szczególnie w zadaniach wymagających odwzorowania całych trajektorii rozwiązań oraz złożonych zależności przestrzenno–czasowych. Wyniki potwierdziły, że modelowanie odwzorowań pomiędzy funkcjami stanowi bardziej naturalne podejście do rozwiązywania równań różniczkowych niż uczenie odwzorowań pomiędzy skończonymi wektorami danych.

Spośród analizowanych architektur najlepsze rezultaty uzyskał operator Fouriera (FNO), który charakteryzował się najwyższą dokładnością i stabilnością zarówno dla równań zwyczajnych, jak i cząstkowych. Architektury DeepONet również przewyższały klasyczny perceptron wielowarstwowy, zachowując poprawną strukturę rozwiązań oraz zdolność do generalizacji na nowe przypadki. Otrzymane wyniki wskazują zatem, że operatory neuronowe stanowią skuteczne narzędzie do aproksymacji operatorów różniczkowych i mogą być wykorzystywane jako alternatywa dla klasycznych architektur sieci neuronowych w problemach opartych na danych funkcyjnych.

Perspektywy badawcze

Wyniki pracy pokazują, że wykorzystanie architektur uwzględniających strukturę danych funkcyjnych pozwala efektywniej wydobywać informacje z dużych zbiorów danych generowanych przez procesy dynamiczne, co czyni operatory neuronowe obiecującym narzędziem dla współczesnych zastosowań analitycznych i predykcyjnych.

Naturalnym kierunkiem rozwoju przedstawionych badań byłoby przetestowanie odporności modeli na niezmienniczość względem siatki (ang. *mesh invariance*), rozszerzenie analizy o bardziej złożone systemy fizyczne (np. równania *Naviera-Stokesa*) oraz wdrożenie nowoczesnych architektur typu *Transformer* czy grafowych sieci neuronowych, które mogą oferować jeszcze większą elastyczność w modelowaniu operatorów nielokalnych.

Bibliografia

[1] George Em Karniadakis i in. “Physics-informed machine learning”. W: *Nature Reviews Physics* (2021).

[2] Zongyi Li i in. “Neural operator: Graph kernel network for partial differential equations”. W: *arXiv preprint arXiv:2003.03485* (2020).

[3] Lu Lu, Pengzhan Jin i George Em Karniadakis. “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators”. W: *Nature Machine Intelligence* (2021).

[4] Zongyi Li i in. “Fourier neural operator for parametric partial differential equations”. W: *ICLR* (2021).

[5] Walter Rudin. *Functional Analysis*. McGraw-Hill, Inc., 1991.

[6] Michał Majchrowski. *Równania różniczkowe cząstkowe i ich zastosowania – Wykład 10: Informacja o przestrzeniach Hilberta*. Politechnika Warszawska, materiały dydaktyczne. Definicja przestrzeni Hilberta, s. 88. 2019. URL: https://pages.mini.pw.edu.pl/~majchrowskim/konw/wykl_10.pdf.

[7] Lawrence C. Evans. *Partial Differential Equations*. American Mathematical Society, 2010.

[8] Volker John. *Numerical Partial Differential Equations – Lecture Notes*. Freie Universität Berlin, WIAS Berlin. 2019. URL: https://www.wias-berlin.de/people/john/LEHRE/NUM_PDE_FUB_19/num_pde_fub_3.pdf.

[9] Daryoush Behmardi i Ensieh Dehghan Nayeri. “Introduction of Fréchet and Gâteaux Derivative”. W: *Applied Mathematical Sciences* 2.20 (2008), s. 975–980. URL: https://www.m-hikari.com/ams/ams-password-2008/ams-password17-20-2008/behmardiAMS17-20-2008.pdf.

[10] Rafał Latała. *Wykład 11 – Analiza Funkcjonalna*. Uniwersytet Warszawski, Wydział Matematyki, Informatyki i Mechaniki, materiały dydaktyczne. 2020. URL: https://www.mimuw.edu.pl/~rlatala/testy/AF/wyklad11.pdf.

[11] George Cybenko. “Approximation by Superpositions of a Sigmoidal Function”. W: *Mathematics of Control, Signals and Systems* 2 (1989), s. 303–314. DOI: 10.1007/BF02551274.

[12] Kurt Hornik. “Approximation Capabilities of Multilayer Feedforward Networks”. W: *Neural Networks* 4.2 (1991), s. 251–257. DOI: 10.1016/0893-6080(91)90009-T. URL: https://web.njit.edu/~usman/courses/cs677/hornik-nn-1991.pdf.

[13] Lu Lu, Pengzhan Jin i George Em Karniadakis. “DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators”. W: *arXiv preprint arXiv:1910.03193* (2021).

[14] A. N. Kolmogorov. “On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition”. W: *Doklady Akademii Nauk SSSR* 114.5 (1957), s. 953–956.

[15] V. I. Arnold. “On the representation of continuous functions of several variables by superposition of continuous functions of one variable”. W: *Doklady Akademii Nauk SSSR* 114 (1957), s. 679–681.

[16] Erwin Kreyszig. *Introductory Functional Analysis with Applications*. John Wiley i Sons. Inc., 1978, s. 82–83.

[17] Zongyi Li i in. “Fourier Neural Operator for Parametric Partial Differential Equations”. W: *arXiv preprint arXiv:2010.08895* (2020).

[18] James D. Murray. *Mathematical Biology: I. An Introduction*. Springer-Verlag, 2002, s. 2–5.

[19] David Romer. *Makroekonomia zaawansowana*. Wydawnictwo Naukowe PWN, 2000, s. 58–65.

[20] Jean Kossaifi i in. “A Library for Learning Neural Operators”. W: *arXiv preprint arXiv:2412.10354* (2025).

Spis tabel

4.1 Przestrzeń hiperparametrów analizowana dla modelu wzrostu logistycznego. . . 28
4.2 Najlepsze konfiguracje hiperparametrów dla modelu wzrostu logistycznego. . . 28
4.3 Błąd średniokwadratowy (MSE) na zbiorze testowym dla modelu wzrostu logistycznego. 29
4.4 Błąd średniokwadratowy (MSE) na danych testowych dla analizowanych architektur sieci neuronowych – model RCK. 31
4.5 Przestrzeń hiperparametrów analizowana dla modelu RCK. 32
4.6 Najlepsze konfiguracje hiperparametrów dla modelu RCK. 32
4.7 Przestrzeń hiperparametrów analizowana dla równania Burgersa. 36
4.8 Najlepsze konfiguracje hiperparametrów dla równania Burgersa. 37
4.9 Względny błąd normy L^2 na zbiorze testowym dla analizowanych architektur w zadaniu rozwiązania równania Burgersa. 37

Spis rysunków

1 Przykład funkcji z analizowanej rodziny. Parametry: $a = 1, b = 0.4, c = 2, d = 0$. 6

2 Krzywe uczenia modelu MLP. Błąd treningowy systematycznie maleje, natomiast błąd walidacyjny pozostaje na zbliżonym poziomie, co wskazuje na przeuczenie w kontekście konkretnej siatki. 7

3 Porównanie rzeczywistej oraz przewidywanej pochodnej dla wybranych próbek testowych. 7

1.1 Hierarchia przestrzeni funkcyjnych wykorzystywanych do reprezentacji ciągłych danych wejściowych. 10

1.2 Kula jednostkowa w różnych normach L^p na $\mathbb{R}^2$, ilustrująca jak różne metryki karzą odchylenia błędów predykcji. 11

1.3 Interpretacja twierdzenia Kołmogorowa jako jednokierunkowej sieci neuronowej, ilustrująca redukcję złożoności poprzez dekompozycję wymiarów. 13

1.4 Dyskretyzacja funkcji ciągłej (a) na siatkach o niższej (b) i wyższej (c) rozdzielczości. 14

1.5 Przykłady strukturyzowanych siatek: regularna (a), prostoliniowa (b) i krzywoliniowa (c). 15

3.1 Schemat architektury DeepONet. Sieć *Branch* przetwarza zdyskretyzowaną funkcję wejściową, generując współczynniki b_k, podczas gdy sieć *Trunk* buduje funkcje bazowe $\tau_k(y)$ zależne od współrzędnych dziedziny. 21

4.1 Niezdolność sieci MLP do generalizacji: model generuje niemal identyczną trajektorię dla różnych danych wejściowych. 29

4.2 Wizualizacja predykcji sieci FNO: wysoka adaptacyjność do zmiennych warunków początkowych. 30

4.3 Zbiorcze zestawienie statystyk dynamiki wzrostu populacji dla wszystkich analizowanych architektur. 31

4.4 Zestawienie predykcji trajektorii oraz błędów bezwzględnych w modelu RCK. Wiersze odpowiadają analizowanym próbkom testowym (33, 209, 280), natomiast kolumny przedstawiają wyniki dla poszczególnych architektur, kolejno: MLP i FNO. 33
4.5 Zestawienie predykcji trajektorii oraz błędów bezwzględnych w modelu RCK. Wiersze odpowiadają analizowanym próbkom testowym (33, 209, 280), natomiast kolumny przedstawiają wyniki dla poszczególnych architektur, kolejno: DeepONet i DeepONet z konwolucją. 34
4.6 Statystyka zbiorcza dynamiki wzrostu kapitału k_t oraz konsumpcji c_t w modelu RCK. Wykresy przedstawiają porównanie średniej predykcji oraz odchylenia standardowego z prawdziwymi wartościami. 35
4.7 Przebieg procesu uczenia analizowanych modeli na zbiorze treningowym i walidacyjnym dla równania Burgersa. 38
4.8 Porównanie rzeczywistego rozwiązania, predykcji oraz map błędu bezwzględnego dla architektury MLP. 39
4.9 Porównanie rzeczywistego rozwiązania, predykcji oraz map błędu bezwzględnego dla architektury DeepONet. 40
4.10 Porównanie rzeczywistego rozwiązania, predykcji oraz map błędu bezwzględnego dla DeepONet z warstwami konwolucyjnymi. 41
4.11 Porównanie rzeczywistego rozwiązania, predykcji oraz map błędu bezwzględnego dla architektury FNO. 42
4.12 Średnia dynamika ewolucji fal w czasie. 44

Streszczenie

Niniejsza praca magisterska poświęcona jest aproksymacji operatorów matematycznych z wykorzystaniem sieci neuronowych. Celem pracy była analiza możliwości zastosowania operatorów neuronowych do przewidywania rozwiązań równań różniczkowych oraz porównanie ich skuteczności z klasyczną architekturą sieci MLP. W części teoretycznej przedstawiono podstawy matematyczne związane z przestrzeniami funkcyjnymi, operatorami matematycznymi, twierdzeniem o uniwersalnej aproksymacji oraz reprezentacją danych funkcyjnych. Omówiono również dwie nowoczesne architektury operatorowe: FNO oraz DeepONet. Część praktyczna obejmowała wygenerowanie zbiorów danych dla modelu wzrostu logistycznego, modelu wzrostu RCK oraz równania Burgersa. Dla każdego problemu wytrenowano cztery modele: MLP, FNO, DeepONet oraz DeepONet z warstwą konwolucyjną. Przeprowadzone eksperymenty wykazały przewagę operatorów neuronowych nad klasycznym MLP, szczególnie w zadaniach wymagających odwzorowania zależności pomiędzy funkcjami. Najlepsze wyniki uzyskała architektura FNO, osiągając najwyższą dokładność i stabilność predykcji.

Słowa kluczowe: uczenie głębokie, operator neuronowy, Fourier Neural Operator, Deep Operator Network, równania różniczkowe, dyskretyzacja.